

Small Radio Telescope Drive Controller

Technical Documentation and Operations Manual

Version 2.0



The 3 m telescope on its alt-azimuth mount at Acre Road Observatory,
with the 1420 MHz feed at prime focus.

Acre Road Observatory
University of Glasgow

Document built 2026-08-27
Describes commit 108eed4 of 2026-08-27

Abstract

This document is the definitive technical description of the Small Radio Telescope (SRT) drive and control system used for 21 cm hydrogen line observations at Acre Road Observatory, Glasgow. The system comprises an Arduino Due carrying out real-time motor control, a WT32-ETH01 (ESP32) controller performing astronomical coordinate transformation, pointing correction and network service, a GNU Radio receiver, and a Flask observation scheduler that also runs the pointing calibration. The document covers the hardware, the firmware on both microcontrollers, the coordinate and pointing mathematics in full, the resident pointing model and the Sun-scan calibration that determines it, the private Ethernet link between the observatory computer and the controller, the receiver and scheduler, and the installation, operation and fault-finding procedures.

Contents

1	Introduction	6
1.1	Purpose and scope	6
1.2	System overview	6
1.3	Two coordinate frames	6
1.4	Document organisation	7
2	System architecture	7
2.1	Architectural overview	7
2.2	Data flow	8
2.3	Communication interfaces	8
3	Hardware configuration	8
3.1	Arduino Due microcontroller	8
3.1.1	Pin assignments	9
3.2	Motor drive system	9
3.2.1	H-bridge motor drivers	9
3.2.2	Position encoders	10
3.2.3	Current sensing	10
3.3	ESP32 controller	10
3.3.1	WT32-ETH01 to Arduino Due wiring	10
3.4	Operational limits	11
4	Arduino Due firmware	11
4.1	Firmware overview	11
4.2	State machine	12
4.3	Homing sequence	13
4.4	Motion profiles	13
4.4.1	Acceleration ramp	13
4.4.2	Deceleration ramp	13
4.5	Stall and limit handling	14
4.6	Serial command interface	14
4.6.1	Configurable parameters	15
4.7	Status output format	15
4.8	Fault detection	16
4.9	Simulation mode	16

5	ESP32 controller	16
5.1	Software architecture	16
5.2	Settings	17
5.3	Networking	18
5.3.1	Network modes	18
5.3.2	Clock discipline	18
5.3.3	DNS resilience	19
5.4	HTTP API	19
5.5	Stellarium protocol	19
5.6	Tracking	20
5.6.1	Below-horizon handling	20
5.6.2	Azimuth wrap-around and the upper stop	20
5.6.3	Galactic-plane target	20
6	Coordinate transformations	21
6.1	Reference frame	21
6.2	Conversion pipeline	21
6.2.1	Julian date	21
6.2.2	Sidereal time	21
6.3	Precession	21
6.4	Horizontal coordinates	22
6.5	Galactic coordinates	22
6.6	Solar position	22
6.7	Lunar position	22
6.8	Accuracy of the transforms	22
7	Pointing model and calibration	23
7.1	Atmospheric refraction	23
7.2	The model	23
7.3	The forward and inverse transforms	24
7.4	Guards	24
7.5	Where the calibration lives	24
7.6	Measuring the model: the Sun scan	25
7.6.1	Raster geometry	25
7.6.2	Beam fit	25
7.6.3	The datum: an absolute (T, D) pair	25
7.7	Fitting the model	25
7.7.1	Design matrix	25
7.7.2	Weighted solution	26
7.7.3	Geometry and quality gates	26
7.7.4	The model in force	27
7.8	Drift scans on the drive grid	27
7.9	Homing first, and the encoder count check	27
7.10	Wire format and application	28
7.11	Operating procedure: a calibration day	28
8	Observatory host and the controller link	29
8.1	What the link is	29
8.2	Why	29
8.3	Address plan	29
8.4	Hardware	29
8.5	Building the link	30

8.6	Firewall	30
8.7	Verification	31
8.8	Recovery	31
8.9	What the link deliberately does not do	32
9	Beam, radiometry and velocity frames	32
9.1	Beam and aperture	32
9.1.1	The measured beam, and how well it is known	33
9.2	System temperature	33
9.3	The radiometer equation	33
9.3.1	Total-power instability	34
9.4	Temperature calibration	34
9.4.1	Single-point blank-sky calibration	34
9.4.2	Noise diode and the Y-factor	34
9.4.3	The automated gain fit against the survey	35
9.4.4	Fitting a drift scan: total power against the model curve	35
9.5	Velocity frames — what is recorded, and what the plots draw	36
9.5.1	The receiver clock	37
10	H1 receiver and observation scheduler	37
10.1	Receiver overview	37
10.1.1	The fixed instrument	37
10.1.2	Overflows	38
10.2	Recordings: where they go and what they are called	38
10.3	HDF5 output format	39
10.4	Observation scheduler	40
10.4.1	Interface	40
10.4.2	Runtime behaviour	41
10.4.3	Coordinate systems and drift scans	43
10.4.4	The measured horizon, in the schedule	43
10.4.5	The horizon scan	43
10.4.6	Observation parameters	44
10.4.7	Automated workflow	45
10.4.8	Tests	45
11	Sky simulator	46
11.1	Sky models	46
11.2	Instrument model	46
11.3	Velocity frames	47
11.4	Modes	47
11.5	The clock, the horizon and the map	47
11.6	Schedule (formerly Realise)	47
12	Installation	48
12.1	Prerequisites	48
12.2	Arduino Due drive firmware	48
12.3	WT32-ETH01 controller firmware	48
12.4	Configuration	48
12.5	Receiver prerequisites	49

13 Testing and verification	49
13.1 Drive firmware in simulation	49
13.2 Native unit tests	49
13.3 Scheduler and calibration tests	49
13.4 The Due emulator	50
14 Operation	50
14.1 Startup sequence	50
14.2 Reaching the web interface	51
14.3 Observing	51
14.4 Safety features	51
15 Troubleshooting	51
15.1 System does not start	51
15.2 Motors do not move	51
15.3 Position errors	52
15.4 Pointing errors	52
15.5 Network and web interface	52
15.6 Coordinates and Stellarium	52
16 Outstanding work and known limitations	53
16.1 Known limitations	53
16.2 Where the open work is recorded	53
17 The 21 cm line: expected signals and the temperature scale	54
17.1 The hyperfine transition	54
17.2 What the line measures	54
17.3 Galactic rotation and the tangent-point method	55
17.4 What the telescope collects	55
17.5 Expected antenna temperatures	56
17.6 Why only the Sun and Moon can calibrate a 5° beam	56
17.7 Expected system temperature	58
17.8 Measuring the system temperature	58
17.8.1 Method A: ambient absorber	58
17.8.2 Method B: the Sun, of known flux density	58
17.8.3 The two-point calibration: blank sky and the Sun as loads	59
17.8.4 Method C: cold sky and the survey slope	61
17.8.5 Recommended procedure	61
18 The codebase	62
18.1 Repository and licence	62
18.2 Working on GitHub	63
18.3 Layout	63
18.4 Root and drive firmware	63
18.5 Controller firmware	64
18.6 Receiver, scheduler and calibration	65
18.7 Sky simulator	66
18.8 Documentation, enclosure and tools	68
18.9 What is deliberately not tracked	68
19 Conclusion	68
A Pin quick reference	69

B	Command quick reference	69
C	HTTP API quick reference	70
C.1	Controller API	70
C.1.1	Status and information	70
C.1.2	Telescope control	70
C.1.3	Pointing, time, network and settings	70
C.2	Scheduler API	71
C.3	Examples	72
	Index	74

1 Introduction

1.1 Purpose and scope

The SRT drive controller is a complete control system for an alt-azimuth mounted radio telescope observing at 1420.405 MHz. It supports:

- Real-time telescope positioning from a web interface
- Integration with planetarium software (Stellarium)
- Conversion between celestial and horizontal coordinates, with atmospheric refraction and a fitted mount pointing model applied
- Measurement of the pointing model from raster scans of the Sun
- Scheduled observations with automatic data acquisition
- Sun, Moon, galactic-plane and satellite tracking

This document describes the system as it now stands. Where a design decision is non-obvious the reasoning is given, because several of them were reached the hard way and are easy to undo by accident.

1.2 System overview

The system separates concerns across four processing units and one host computer:

1. **Arduino Due** — low-level motor control, encoder counting, limit-switch homing, current sensing and fault handling. It works exclusively in *drive* coordinates.
2. **WT32-ETH01 controller** — network interface, astronomical transforms, the resident pointing model, web server and Stellarium protocol. It is the boundary between the sky and the mount.
3. **H1 receiver** — GNU Radio spectrometer for 21 cm data acquisition.
4. **Observation scheduler** — Flask application coordinating pointing and recording, and hosting the Sun-scan calibration.
5. **Observatory computer** — runs the receiver and the scheduler, and owns a private Ethernet link to the controller on which it provides the address, DNS and a route to the internet.

A fifth component, the **sky simulator** (Section 11), is not part of the observing chain but predicts what the instrument should see and can hand a simulated pointing to the real telescope.

1.3 Two coordinate frames

One distinction runs through the whole system and is worth stating before anything else.

True alt/az

Where the dish looks on the sky. Every astronomical quantity is computed in this frame, using the true site latitude and longitude.

Drive

What the mount mechanically is: encoder pulses counted from the limit switches, quantised to 0.5° . The Arduino Due works only in this frame.

The two are joined by the transform in Section 7, which applies atmospheric refraction and the fitted pointing model. Every commanded position crosses from true to drive on its way out; every *measured* position crosses from drive to true on its way back. Arrival and slew-completion tests are made wholly within the drive frame, so the two never mix.

1.4 Document organisation

Section 2 presents the system architecture; Section 3 the hardware; Section 4 the Arduino Due firmware; Section 5 the ESP32 controller; Section 6 the coordinate transformations; Section 7 the pointing model and its calibration; Section 8 the observatory host and the private controller link; Section 9 the beam, the radiometry and the velocity frames; Section 10 the receiver and scheduler; Section 11 the sky simulator; Section 12 installation; Section 13 testing and verification; Section 14 operation; Section 15 fault-finding; Section 16 the work still open; Section 17 the astronomy of the 21 cm line, the signal strengths this telescope should see, and how to measure its system temperature from the sky and the Sun; and Section 18 the repository and the purpose of every file in it.

2 System architecture

2.1 Architectural overview

Figure 1 shows the components and the paths between them.

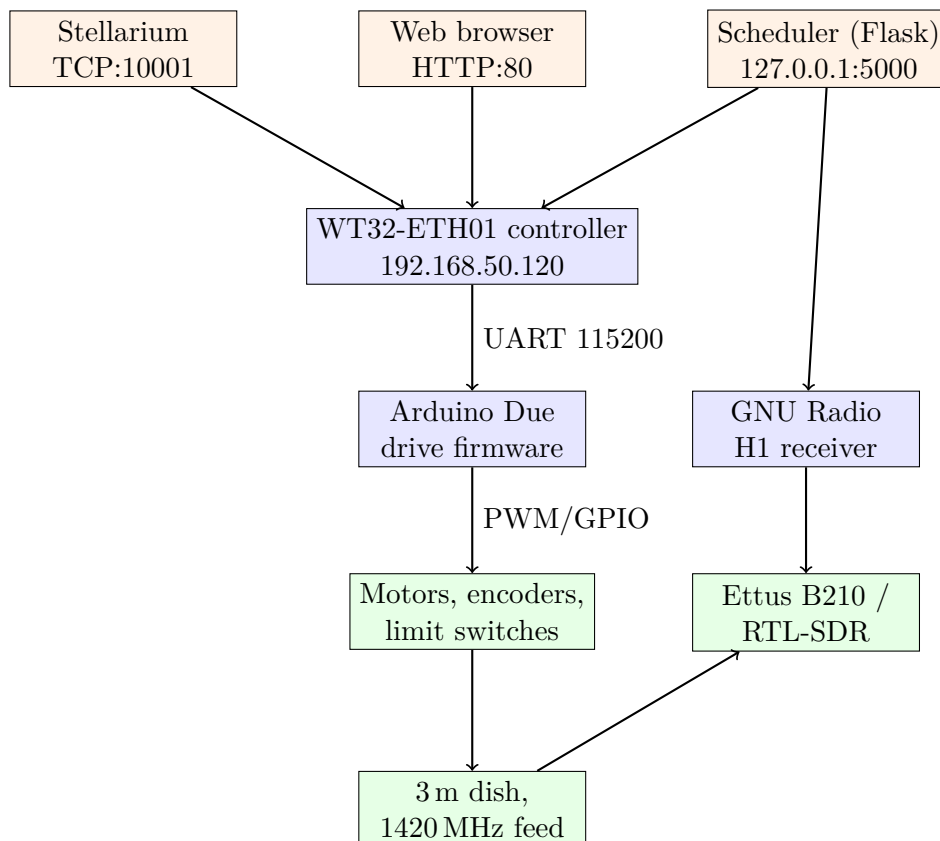


Figure 1: System architecture. Stellarium, the browser and the scheduler all run on the observatory computer, which reaches the controller over the private Ethernet link of Section 8.

2.2 Data flow

A typical observation proceeds as follows:

1. The user or scheduler specifies a target (RA/Dec, galactic, alt/az, solar-system object, or TLE).
2. The controller converts the target to *true* alt/az for the current sidereal time.
3. The observing horizon is tested against the true altitude.
4. `trueToDrive()` applies refraction and the pointing model, producing a *drive* position.
5. The drive position is tested and clamped against the mechanical mount limits, then sent to the Due over UART.
6. The Due computes a motion profile and drives the motors under PWM; reed-switch encoders count position.
7. The Due reports drive position, motor currents and status once a second.
8. The controller publishes the drive position as it stands, and its `driveToTrue()` image as the sky position, from which RA/Dec and galactic coordinates are derived.
9. Once the slew has completed, the scheduler starts the receiver and data is written to HDF5.

2.3 Communication interfaces

Table 1: System communication interfaces

Interface	Protocol	Speed	Purpose
Due ↔ controller	UART (Serial1)	115200 baud	Drive targets, status
Due ↔ USB	Serial	115200 baud	Debug, direct control
Controller ↔ browser	HTTP	TCP:80	Web interface, REST API
Controller ↔ Stellarium	TCP	Port 10001	Telescope protocol
Controller ↔ scheduler	HTTP	TCP:80	Pointing, status, model push
Controller Ethernet	100BASE-TX	LAN8720 RMII	Private link to the host
Controller WiFi	IEEE 802.11	—	AP fallback at 192.168.4.1
Controller OTA	ArduinoOTA	UDP/TCP 3232	Firmware update over Ethernet
Scheduler ↔ browser	HTTP	127.0.0.1:5000	Scheduler UI and API

The scheduler binds to loopback only. It has no authentication and its endpoints start observations, claim the SDR, rewrite the schedule, push a pointing model and flash controller firmware; exposed on a wildcard address it would be an unauthenticated telescope-control API. Nothing needs the wildcard — the controller’s own page fetches the scheduler at 127.0.0.1:5000 from a browser on the same host, and remote use is over `waypipe`.

3 Hardware configuration

3.1 Arduino Due microcontroller

The Arduino Due is the real-time motor control platform, chosen for its ARM Cortex-M3 core at 84 MHz with deterministic interrupt handling, its 12-bit ADCs, multiple hardware UARTs, adequate PWM resolution and 3.3 V logic compatible with the ESP32.

3.1.1 Pin assignments

Table 2: Arduino Due pin assignments

Function	Pin	Wire colour	Notes
<i>Fault flags (inputs)</i>			
Az fault flag 1	5	Blue	Driver status
Az fault flag 2	4	Purple	Driver status
Alt fault flag 1	7	Blue	Driver status
Alt fault flag 2	6	Purple	Driver status
<i>Motor control (outputs)</i>			
Az PWM	8	Green	Speed control (inverted)
Az direction	9	Yellow	Sense inverted in software
Alt PWM	10	Green	Speed control (inverted)
Alt direction	11	Yellow	Sense inverted in software
<i>Driver reset (outputs)</i>			
Az reset	22	Black	HIGH = enabled
Alt reset	24	White	HIGH = enabled
<i>Position encoders (inputs)</i>			
Az pulses	12	Yellow	Reed switch, falling edge
Alt pulses	13	Blue	Reed switch, falling edge
<i>Current sensors (analogue inputs)</i>			
Az current	A1	Black	Hall effect
Alt current	A0	White	Hall effect
<i>Serial1 (controller interface)</i>			
TX1	18	—	To controller RX (IO14)
RX1	19	—	From controller TX (IO4)
<i>Calibrator (output)</i>			
Calibrator control	26	—	Active HIGH, noise source

The direction sense of *both* axes is inverted in software, by the `AZ_DIR_INVERT` and `ALT_DIR_INVERT` macros in `include/config.h`. Every read or write of `PIN_DIR_AZ` and `PIN_DIR_ALT` must therefore go through the `AZ_DIR()` and `ALT_DIR()` macros; a direct `digitalWrite` to those pins will drive the wrong way.

3.2 Motor drive system

3.2.1 H-bridge motor drivers

Each axis is driven by an H-bridge module with:

- Two fault flag outputs (FF1, FF2) encoding the fault type (Table 9)
- An active-low reset input, held HIGH to enable the driver
- Inverted PWM speed control: 255 is stopped, 0 is full speed
- A single direction input

The fault outputs are electrically noisy at motor start and stop, because of inductive kick-back and supply sag. `checkFaultFlags()` therefore requires the same fault to be present on `FAULT_FLAG_PERSIST_COUNT` (5) consecutive calls before reporting it.

3.2.2 Position encoders

Position feedback comes from reed-switch encoders on each axis:

- Resolution `PULSES_PER_DEGREE` = 2, i.e. a 0.5° quantum
- Falling-edge interrupts; direction is inferred from the static direction pin at the moment of the pulse
- Debounce windows of 50 ms (azimuth) and 100 ms (altitude), both settable

The debounce timestamp is updated *only* on pulses that pass the window. Updating it on every edge — the original implementation — caused total pulse loss as the motor speed approached the debounce interval, because every bounce reset the timer and no edge ever counted. This must not be reintroduced.

3.2.3 Current sensing

Motor current is measured by hall-effect sensors read through the Due's 12-bit ADC against its 3.3 V reference. The sensors are bipolar with a mid-rail zero, so

$$V = \frac{V_{\text{ref}}}{2^{12}} N_{\text{ADC}}, \quad I = \frac{V - V_{\text{offset}}}{S}, \quad (1)$$

with $V_{\text{ref}} = 3.3 \text{ V}$, a nominal $V_{\text{offset}} = 1.65 \text{ V}$ and sensitivity $S = 0.044 \text{ V/A}$.

Two refinements matter. First, the reported current is a single-pole low-pass filter of the raw reading,

$$I_k = I_{k-1} + \alpha (I_k^{\text{raw}} - I_{k-1}), \quad \alpha = 0.02, \quad (2)$$

and it is the *filtered* value, never the raw one, that `checkCurrentLimits()` tests — otherwise a single ADC excursion trips an overcurrent fault. Second, the zero offset is re-tracked continuously while an axis is idle,

$$V_k^{\text{offset}} = V_{k-1}^{\text{offset}} + \beta (V_k - V_{k-1}^{\text{offset}}), \quad \beta = 0.005, \quad (3)$$

so long-term drift of the hall sensor settles towards zero by itself. The offsets are additionally measured directly at startup, by averaging 50 samples with the motors off.

3.3 ESP32 controller

The WT32-ETH01 is an ESP32 board with a native LAN8720A Ethernet PHY on RMII as well as WiFi. It is the deployed board and the canonical build target.

3.3.1 WT32-ETH01 to Arduino Due wiring

IO4 and IO14 are used for the Due link, keeping TX0/RX0 free for the first serial flash and for recovery.

Table 3: WT32-ETH01 to Arduino Due serial connection

WT32-ETH01	Arduino Due	Function
IO4	Pin 19 (RX1)	Controller TX → Due RX
IO14	Pin 18 (TX1)	Controller RX ← Due TX
GND	GND	Common ground
5V	5V	Power

Table 4: WT32-ETH01 first-flash connection (temporary FT232 programmer)

FT232 programmer	WT32-ETH01	Function
TXD	RX0 (GPIO3)	Programmer TX → ESP32 RX
RXD	TX0 (GPIO1)	Programmer RX ← ESP32 TX
GND	GND	Common ground

The programmer is temporary: its power, IO0, EN, DTR and RTS wiring must not remain connected in normal operation. After one OTA-capable serial flash, routine updates go over Ethernet with the `wt32-eth01-ota` environment.

A printed enclosure for the board is in `enclosure/`, as an OpenSCAD source with the board dimensions and wall thicknesses as named parameters, plus exported STLs for the base and lid. It is designed to mount on the outside of a larger box with a 15 mm wire feedthrough, and the Ethernet jack sets the internal height.

3.4 Operational limits

Table 5: Position limits. The hardware limits are the physical switches; the software limits are the operating envelope inside them.

Axis	Minimum	Maximum
Altitude (hardware)	0° (limit switch)	90° (zenith)
Altitude (software)	0°	90°
Azimuth (hardware)	0° (limit switch)	355° (limit switch)
Azimuth (software)	2°	353°

The software azimuth limits sit 2° inside the physical stops. A further `SOFTWARE_LIMIT_TOLERANCE` of 4° is allowed as a transient excursion before the firmware forces a stop, but never past a hardware limit.

These are all *drive*-frame quantities. The separate observing horizon (Section 5) is a true-frame quantity and is a different thing entirely.

4 Arduino Due firmware

4.1 Firmware overview

The drive firmware (`src/main.cpp`) is a single bare-metal translation unit implementing:

- An automatic three-phase homing sequence at power-on
- Ramped motion profiles that prevent structural oscillation

- Interrupt-driven encoder position tracking
- Safety monitoring: overcurrent, stall, limit switches, driver fault flags
- A line-oriented serial command interface on both ports
- Configuration persistence to flash via DueFlashStorage

Position is held in encoder pulses. Displayed degrees are position/PULSES_PER_DEGREE, and pulse position 0 is the hardware lower limit on each axis. After homing, both axes read (0,0).

4.2 State machine

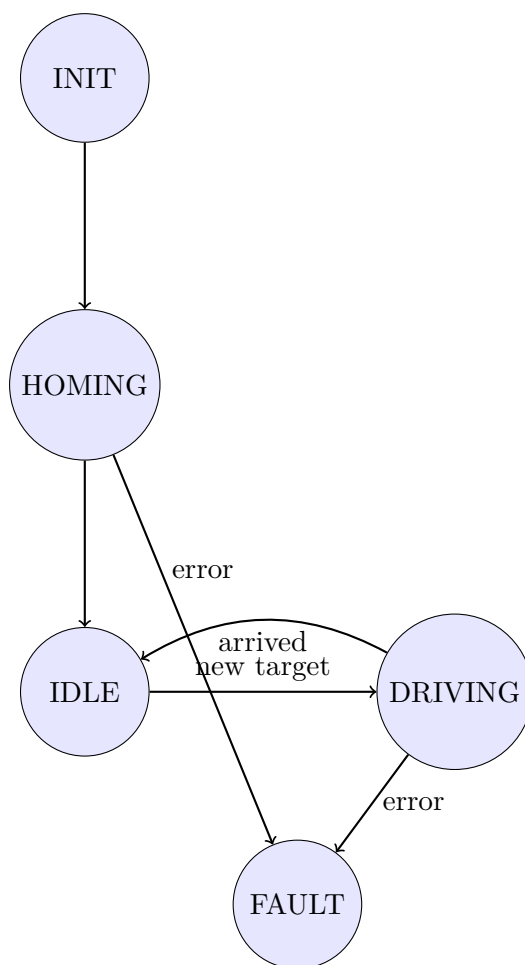


Figure 2: Drive firmware state machine

INIT Hardware initialisation, GPIO configuration, current-sensor zeroing, driver enable. Reported as **Initializing**.

HOMING Running the homing sequence. Reported as **Homing**.

IDLE Accepting commands. Reported as **Ready**.

DRIVING Slewing. Reported as **Slewing**, or **Reversing** while an axis is decelerating to change direction.

FAULT Motors disabled; reported as **FAULT** with the fault text in brackets.

The motion controller `updateAxisMotion()` runs from the main loop except in HOMING and FAULT. The IDLE→DRIVING transition fires *only* when a new-target flag has been set by `executeDrive()`. Without that condition the controller would chase stray encoder pulses while idle and could walk itself into a limit switch.

4.3 Homing sequence

Homing establishes the encoder origin in three phases, because a single fast run into the stop leaves an origin that depends on how fast the axis was travelling when it hit.

1. **Drive to limits.** Both axes ramp up to full speed and run negative into the hard stops. An axis is judged to have arrived when no encoder pulse has been seen for `stallTimeoutMs` (2s by default).
2. **Back off.** Both axes reverse 5° under the same ramp profile.
3. **Re-approach.** Both axes run into the stops again over the short remaining distance, so the approach is slow and the zero is repeatable.

Both helpers obtain their profile from `calculatePWM(INT32_MAX, startTime)`, which yields the standard ramp-up without ever entering the ramp-down branch. After phase 3 both position counters are set to zero. The mount then drives to the configured home offset,

$$n_{\text{home}} = \text{cfg.homeAz} \times \text{PULSES_PER_DEGREE}, \quad (4)$$

computed directly from the home angle — the software minimum is *not* subtracted — and the position counters are set to the home coordinates. Both `homeAlt` and `homeAz` default to 0°, so by default the mount finishes homing sitting on its origin. A fault flag, overcurrent condition or unresponsive motor at any point aborts homing into FAULT. The whole sequence takes 30–60s depending on the starting position.

4.4 Motion profiles

PWM values are inverted: PWM = 255 is stopped, PWM = 0 is full speed, and `PWM_MIN_SPEED` = 100 is the slowest usable drive. In the expressions below a *smaller* number is a faster motor.

4.4.1 Acceleration ramp

For the first T_{ramp} milliseconds of a slew (`RAMPUP`, default 500 ms) the PWM value interpolates linearly from the minimum speed to full speed:

$$\text{PWM}(t) = \text{PWM}_{\text{min}} - \frac{t}{T_{\text{ramp}}}(\text{PWM}_{\text{min}} - \text{PWM}_{\text{full}}) = 100 \left(1 - \frac{t}{T_{\text{ramp}}}\right). \quad (5)$$

Thereafter the axis cruises at $\text{PWM}_{\text{full}} = 0$.

4.4.2 Deceleration ramp

Deceleration is keyed to the number of encoder pulses remaining, n , and begins when n falls below $n_{\text{ramp}} = \text{RAMPDOWN} \times \text{PULSES_PER_DEGREE}$ (default 7°, so 14 pulses):

$$\text{PWM}(n) = \min(\text{PWM}_{\text{min}}, \max(\text{PWM}_{\text{full}}, 218 - n^2)). \quad (6)$$

The quadratic in n gives an approximately constant deceleration in position; the clamp at PWM_{min} stops the axis creeping so slowly in the last degree that the encoder debounce swallows its pulses. `STOPRAMP` (default 300 ms) is the corresponding deceleration time applied before an axis reverses direction.

4.5 Stall and limit handling

A stall is declared only when *both*

$$t - t_{\text{drive start}} > T_{\text{stall}} \quad \text{and} \quad t - t_{\text{last pulse}} > T_{\text{stall}} \quad (7)$$

are satisfied. The first condition matters: without it a stale `lastPulse` left over from a long idle period would trigger a false stall the instant a new slew began.

A stall that occurs while driving negative and within 2° of the hardware limit is not a fault — it is the limit switch. The position snaps to the limit value, the target is clamped, and the axis returns to idle. This is how the mount recovers gracefully from a commanded position slightly outside its range.

4.6 Serial command interface

Commands are accepted on the programming-port USB serial and on Serial1 (the controller link), both at 115200 baud.

Table 6: Motion and calibrator commands

Command	Description
<alt> <az>	Slew to position (e.g. 45.0 180.0)
DRIVE <alt> <az>	Slew to position (explicit form)
HOME	Run the homing sequence
STOP	Emergency stop
RESET	Clear fault
CAL ON / CAL OFF / CAL	Noise-source calibrator on, off, toggle

The calibrator is a noise diode used for receiver calibration; `CAL ON` drives GPIO 26 high.

Table 7: Configuration commands

Command	Description
STATUS	Print one status line immediately
CONFIG	Show all configuration parameters
SET <param> <value>	Set a configuration parameter
SAVE	Save configuration to flash
LOAD	Load configuration from flash
DEFAULTS	Reset to compiled defaults
HELP	Show command help

4.6.1 Configurable parameters

Table 8: SET parameters

Parameter	Description	Default	Unit
ALTHWMIN, ALTHWMAX	Altitude hardware limits	0, 90	degrees
AZHWMIN, AZHWMAX	Azimuth hardware limits	0, 355	degrees
ALTMIN, ALTMAX	Altitude software limits	0, 90	degrees
AZMIN, AZMAX	Azimuth software limits	2, 353	degrees
HOMEALT, HOMEAZ	Home position (encoder origin)	0, 0	degrees
RAMPUP	Acceleration time	500	ms
RAMPDOWN	Deceleration distance	7	degrees
STOPRAMP	Reversal deceleration time	300	ms
CURRENT	Overcurrent threshold	5.0	A
STALL	Stall detection timeout	2000	ms
DEBOUNCE	Azimuth encoder debounce	50	ms
DEBOUNCE_ALT	Altitude encoder debounce	100	ms
BACKLASH	Azimuth backlash compensation	0.0	degrees

Configuration lives in a `Config` struct in RAM and is persisted to flash on `SAVE`. `loadDefaults()` must initialise *every* field — a field left out holds garbage until the next `SAVE`. Adding a tunable parameter means five coordinated edits: a `DEFAULT_X` macro in `include/config.h`, a field in the `Config` struct, an initialisation in `loadDefaults()`, a case in `processSetCommand()`, and lines in `showConfig()` and `showHelp()`.

4.7 Status output format

The status line is positional and the controller’s parser depends on it. Do not change the field order or spelling.

```
Alt:%.1f Az:%.1f Ialt:%.1fA Iaz:%.1fA Status:<state> [<fault>] -> Alt:%.1f Az:%.1f Cal:ON|OFF
```

The `[<fault>]` group appears only in `FAULT`, and the `-> Alt: Az:` group only while `DRIVING`.

```
1 Alt:45.0 Az:180.0 Ialt:0.5A Iaz:0.4A Status:Ready Cal:OFF
2 Alt:30.0 Az:150.0 Ialt:2.1A Iaz:1.9A Status:Slewing -> Alt:45.0 Az:180.0 Cal:OFF
3 Alt:30.0 Az:150.0 Ialt:5.2A Iaz:0.0A Status:FAULT [Altitude motor overcurrent]
  Cal:OFF
```

Listing 1: Status output examples

Two independent output disciplines apply.

Programming-port de-duplication

`outputStatus()` builds a comparison string that excludes the live current readings and prints to the USB port only when that string changes. Without it the port would emit a line per main-loop pass with nothing but ADC noise varying.

Serial1 rate limiting

The unsolicited line to the controller is emitted at most once per `STATUS_INTERVAL_MS` (1s). It used to be unconditional, which meant roughly 100 lines per second, about 6.5kB/s, into a receiver that drains five lines a second through a 256-byte buffer. That buffer was in permanent overflow, so bytes were dropped mid-line and the tail of one line

arrived welded to the head of the next — splices that still passed the controller’s format check and parsed into plausible-looking nonsense. A **STATUS** command arriving on Serial1 is answered immediately on its own path and is not rate limited, and that is how the controller actually polls.

A **STATUS** command received on Serial1 is answered only on Serial1, and never disturbs the programming-port de-duplication buffer.

4.8 Fault detection

Table 9: Fault conditions and detection

Fault	Detection	Likely cause
Short circuit	FF1 LOW, FF2 HIGH	Wiring fault, motor failure
Overheating	FF1 HIGH, FF2 LOW	Prolonged high current
Undervoltage	FF1 HIGH, FF2 HIGH	Power supply sag
Overcurrent	Filtered $I > \text{CURRENT}$	Obstruction, binding
Stall	No pulses for STALL	Motor failure, jam, limit
Position bounds	Position outside limits	Encoder error

4.9 Simulation mode

The `simulation` environment overrides `analogWrite`, `digitalWrite`, `analogRead` and `attachInterrupt` with macros, so the firmware runs end to end with no hardware attached. `simulatePulses()` advances position from the commanded PWM and is called both from the main loop and from inside the homing while-loops.

```
1 ~/.platformio/penv/Scripts/pio.exe run -e simulation
```

Any change to the homing flow or to in-loop motor control must keep working in simulation; both the `due` and `simulation` environments must build clean.

Table 10: Simulation parameters (`include/config.h`)

Parameter	Default	Description
<code>SIM_MAX_SPEED_DEG_S</code>	180.0	Maximum simulated speed (deg/s)
<code>SIM_INITIAL_AZ_DEG</code>	20.0	Starting azimuth before homing
<code>SIM_INITIAL_ALT_DEG</code>	10.0	Starting altitude before homing
<code>SIM_STALL_TIMEOUT_MS</code>	200	Faster stall detection in simulation

The current-sensor pins are *not* stubbed: A0 and A1 read the real ADC even in simulation, so current sensing and its calibration can be exercised against real sensors while the motion loop runs simulated.

5 ESP32 controller

5.1 Software architecture

The controller is an Arduino-framework PlatformIO project. `state.h` declares the single shared `SRTState` struct; all subsystems mutate it directly, under a cross-task lock where the mutating task is not `loopTask`.

Table 11: Controller source files

File	Purpose
main.cpp	Application, tracking loop, clock discipline
config.h	Compile-time defaults
settings.cpp/h	Runtime settings, NVS persistence
coordinates.cpp/h	Astronomical transforms
pointing.cpp/h	Pointing model, true/drive transform, refraction
web_server.cpp/h	Async HTTP server and REST API
stellarium.cpp/h	Stellarium telescope protocol
srt_serial.cpp/h	UART link to the Due
wifi_manager.cpp/h	WiFi AP/station management
index_html.h	Embedded web interface

HTTP is served by ESPAsyncWebServer and the Stellarium protocol by AsyncTCP, both on the `async_tcp` task; the tracking loop runs on `loopTask`. State touched by both is guarded by the `SRTLock` scoped lock. Flash writes — settings and the pointing model — are deliberately performed *outside* that lock: they take tens of milliseconds and nothing on `loopTask` writes either structure, so there is no writer to race and no reason to stall tracking.

5.2 Settings

Table 12: Runtime settings and their compiled defaults

Setting	Default	Frame	Meaning
observer_lat	55.902426	—	True site latitude (deg N)
observer_lon	−4.307865	—	True site longitude (deg E)
mount_alt_min/max	0, 90	drive	Mechanical altitude stops
mount_az_min/max	2, 353	drive	Mechanical azimuth stops
horizon_alt	10.0	true	Local observing horizon
galactic_min_alt	45.0	true	Galactic-plane acquisition floor
stow_alt, stow_az	90, 180	drive	Park position
position_deadband	0.25	drive	Minimum move worth commanding
ap_ssid	SRT_Controller	—	WiFi AP name
ap_password	radio1420	—	WiFi AP password
page_name	—	—	Title shown in the web UI

Three distinctions in that table are load-bearing and have each been merged by mistake at least once.

Observing horizon versus mount limits

`horizon_alt` is a question about the sky — has the source cleared the trees — so it is tested against the target’s *true* altitude, before the pointing transform. `mount_alt_min` is a question about the machine, and clamps the *drive* altitude afterwards. They were once merged into a single “effective tracking horizon” equal to the larger of the two, which quietly reinterpreted a mechanical stop as a sky horizon and left neither testable on its own.

Stow versus the Due’s home

`stow_alt/stow_az` is where the dish *parks*. It is the one sky-looking setting held in *drive* coordinates, and the pointing model is deliberately bypassed on the stow path. Parking is mechanical — “leave the mount here” — not an observation, and at the default zenith

stow the distinction is not academic: azimuth is degenerate there, every azimuth points at the same piece of sky, and the model’s $\tan h$ azimuth term asks for a correction of order 50° that would move the beam by exactly nothing. Through the model the dish would park at drive azimuth 170° . The Due’s HOMEALT/HOMEAZ are also drive coordinates but mean something different again: the *encoder origin*, what the limit-switch stall corresponds to.

Operator offset versus calibration

The `/offset` boxes are a deliberate sky-frame displacement layered on top of the calibration — for raster mapping and beam switching — and never a substitute for it. The offset is applied in the true frame, before the model, so “ 0.5° off source” means half a degree on the sky at any elevation.

5.3 Networking

5.3.1 Network modes

1. **Ethernet** — LAN8720 PHY, DHCP by default. This is the operational path; see Section 8.
2. **WiFi access point** — always active at 192.168.4.1, independent of the Ethernet configuration, and the standing fallback.
3. **WiFi station** — connects to a saved network if one is configured.
4. **mDNS and OTA** — discoverable as `srt-controller.local`; firmware upload over Ethernet on port 3232.

Table 13: Default network configuration

Setting	Value
Ethernet address	DHCP (pinned by MAC to 192.168.50.120)
Static-IP fallback	192.168.50.120/24, gateway and DNS 192.168.50.1
WiFi AP SSID / password	SRT_Controller / radio1420
AP address	192.168.4.1
Hostname	srt-controller.local
Web server port	80
Stellarium port	10001
OTA port	3232
NTP server	pool.ntp.org, numeric fallback 162.159.200.123

The static-IP values are never applied in normal operation, because the controller runs on DHCP — but they are what the Static IP form is pre-filled with, so they must name the link the controller is actually on. They were once left at a `192.168.1.x` factory placeholder, which meant one tick of that box would have moved the controller off the link entirely, reachable only over the WiFi AP.

5.3.2 Clock discipline

The lwIP SNTP client polls in the background every hour; nothing blocks waiting for it. Success is declared only by the `onTimeSync()` callback, so a clock that has never been set cannot report itself as synced. `/time/status` publishes `sync_state`, `stale`, `last_sync_age_s` (which is -1 , not 0, when no sync has ever happened), `last_offset_ms` and `sync_count`. A browser-set time

is recorded as source **browser** and deliberately does *not* update the last-sync epoch, so it reports as unverified rather than masquerading as a checked clock.

Clock accuracy is a pointing specification, not a convenience. An error of τ seconds in UTC appears directly as an hour-angle error of 15.041τ arcseconds, so the mount loses half an encoder quantum (0.25°) after only 60 s of clock error, and a degree after four minutes.

5.3.3 DNS resilience

lwIP keeps one process-wide resolver list rather than one per interface, and bringing the WiFi AP up clears it. The firmware caches the resolver address supplied by DHCP and restores it whenever it finds the list empty. `/network` reports `dns_after_eth`, `dns_after_wifi`, `lwip_dns0` and a `dns_restores` counter; `dns_after_wifi` reading 0.0.0.0 and a slowly climbing restore count are both expected, not faults.

5.4 HTTP API

The full endpoint list is in Appendix C. The endpoints that carry frame semantics are worth stating here.

`/status`

`alt` and `az` are the Due's *drive* position — the scheduler's slew-completion check and `sun_scan.py`'s record of where a source was found both need that frame, and both compare against a drive target. `true_alt` and `true_az` are the sky position it corresponds to, and it is those, not the drive position, from which RA/Dec and galactic l , b are computed. Converting the drive position straight to RA/Dec would be wrong by the whole calibration. The response also carries `pointing_loaded`, `clock_state`, `clock_age_s`, and the `lock_timeouts` and `malformed_status` counters, both of which are zero in normal operation.

`/goto`, `/track/*`, `/direct`

Targets are true-frame and are rejected below `horizon_alt` before any state is mutated. They pass through `trueToDrive()` before being sent to the Due.

`/go-home`

Slows to the stow position. This is the documented exception: stow is already a drive position, so it is sent as it stands and merely clamped to the mount limits. `state.targetAlt/targetAz` are deliberately *not* written, because those hold a true-frame target and putting drive coordinates in them would be exactly the frame mix the design exists to prevent. `/home`, by contrast, runs the Due's physical homing sequence.

`/pointing`, `/pointing/apply`, `/pointing/clear`

The resident pointing model; see Section 7.

`/ping`, `/network`

Minimal diagnostics, registered ahead of the full UI handler so they answer even if the main page does not.

5.5 Stellarium protocol

The controller implements the Stellarium telescope protocol on TCP port 10001: a 20-byte `goto` message carrying RA and Dec as unsigned 32-bit fractions, and a 24-byte position report in the same encoding. Reported positions are `driveToTrue()` images of the measured drive position, so the marker in Stellarium is where the dish is looking rather than where the encoders read.

To configure: Configuration → Plugins → Telescope Control, enable and restart, add a telescope of type “External software or remote computer”, host 192.168.50.120, port 10001. Select an object and press Ctrl+1 to slew.

5.6 Tracking

With tracking enabled the controller, once a second:

1. Refreshes the target’s RA/Dec (every 30s for the Sun, Moon and galactic plane; static otherwise)
2. Converts to true alt/az for the current sidereal time
3. Tests the true altitude against `horizon_alt`
4. Applies the operator offset and any axis-hold, in the true frame
5. Converts to the drive frame with `trueToDrive()`
6. Tests and clamps against the mount limits
7. Sends the drive target if it differs from the last one by more than the deadband

5.6.1 Below-horizon handling

When the true altitude falls below `horizon_alt` the dish parks at the stow position and the controller enters “waiting for rise”. Position calculation continues each second, and tracking resumes automatically when the target clears the horizon again.

5.6.2 Azimuth wrap-around and the upper stop

If the drive azimuth falls outside the mount limits — which happens to circumpolar sources crossing the azimuth dead zone — the controller enters “waiting for wrap”, continues computing, and resumes when the target re-enters the range. Note that the rise verdict is settled *before* the transform and the wrap verdict *after* it: clearing the wrap flag in the true frame would announce “wrap complete” every second for a target still mechanically unreachable.

Above `mount_alt_max` the mount is simply left where it is. The alternative — driving to the altitude stop and continuing to track in azimuth — would run the mount along its limit switch for hours to follow a target it cannot point at.

5.6.3 Galactic-plane target

`/track/galactic-plane` walks outwards along $b = 0$ from the galactic centre in 0.5° steps and returns the first longitude whose current altitude reaches `galactic_min_alt`. At each step the two candidates $l = +d$ and $l = 360^\circ - d$ are equidistant from the centre, so the choice between them is free and the higher is taken — it stays observable longer and looks through less atmosphere.

The 45° default floor is an *acquisition* threshold, not a horizon: once acquired the target is followed down until the ordinary observing horizon parks the dish. It is high deliberately. The galactic centre is at $\delta = -28.9^\circ$ and from Glasgow ($\phi = 55.90^\circ$) culminates at

$$h_{\max} = 90^\circ - |\phi - \delta| = 90^\circ - 84.8^\circ = 5.2^\circ, \quad (8)$$

so it never clears the 10° horizon — zero hours in a year. A fixed “galactic bulge” button was therefore permanently in fallback. At a 45° floor some point on the plane qualifies about 73% of the time, and never within about 45° of longitude of the centre.

6 Coordinate transformations

6.1 Reference frame

All equatorial coordinates are J2000.0: standard epoch and equinox J2000.0, equivalent to ICRS to within milliarcseconds, and compatible with modern catalogues and with Stellarium.

6.2 Conversion pipeline

$$\text{RA/Dec (J2000)} \xrightarrow{\text{precess}} \text{RA/Dec (date)} \xrightarrow{\text{HA}} \text{true Alt/Az} \xrightarrow[+ \text{model}]{+ \text{refraction}} \text{drive Alt/Az} \quad (9)$$

The first three steps are described here; the last is Section 7.

6.2.1 Julian date

$$\text{JD} = \lfloor 365.25(Y + 4716) \rfloor + \lfloor 30.6001(M + 1) \rfloor + D + B - 1524.5 + \frac{h + m/60 + s/3600}{24} \quad (10)$$

with Y the year, M the month (January and February counted as months 13 and 14 of the previous year), D the day, $B = 2 - \lfloor Y/100 \rfloor + \lfloor Y/400 \rfloor$, and h, m, s UTC.

6.2.2 Sidereal time

GMST in hours, by the IAU 1982 expression [3]:

$$\text{GMST} = 6.697374558 + 0.06570982441908 D_0 + 1.00273790935 H + 2.6 \times 10^{-5} T^2 \quad (11)$$

where D_0 is days since J2000.0 at 0^h UT, H is hours since 0^h UT, and $T = D_0/36525$. Local sidereal time follows as

$$\text{LST} = \text{GMST} + \lambda/15 \quad (12)$$

with λ the observer's longitude in degrees, east positive.

6.3 Precession

IAU 1976 precession [2] takes J2000 coordinates to the equinox of date. With $T = (\text{JD} - 2451545.0)/36525$ the angles, in arcseconds, are

$$\zeta_A = (2306.2181 + 1.39656 T) T + 0.30188 T^2 + 0.017998 T^3 \quad (13)$$

$$z_A = (2306.2181 + 1.39656 T) T + 1.09468 T^2 + 0.018203 T^3 \quad (14)$$

$$\theta_A = (2004.3109 - 0.85330 T) T - 0.42665 T^2 - 0.041833 T^3 \quad (15)$$

and the rotation is applied through

$$A = \cos \delta_0 \sin(\alpha_0 + \zeta_A) \quad (16)$$

$$B = \cos \theta_A \cos \delta_0 \cos(\alpha_0 + \zeta_A) - \sin \theta_A \sin \delta_0 \quad (17)$$

$$C = \sin \theta_A \cos \delta_0 \cos(\alpha_0 + \zeta_A) + \cos \theta_A \sin \delta_0 \quad (18)$$

giving

$$\alpha = \text{atan2}(A, B) + z_A, \quad \delta = \arcsin C. \quad (19)$$

6.4 Horizontal coordinates

With hour angle $HA = LST - \alpha$ and observer latitude ϕ ,

$$\sin h = \sin \delta \sin \phi + \cos \delta \cos \phi \cos HA, \quad (20)$$

$$\cos A = \frac{\sin \delta - \sin h \sin \phi}{\cos h \cos \phi}, \quad A = \begin{cases} \arccos(\cos A) & \sin HA \leq 0 \\ 2\pi - \arccos(\cos A) & \sin HA > 0. \end{cases} \quad (21)$$

Azimuth is measured from north through east. This yields the *true* alt/az of Section 1.3.

6.5 Galactic coordinates

Using the J2000 galactic pole constants $\alpha_{\text{NGP}} = 192.85948^\circ$, $\delta_{\text{NGP}} = 27.12825^\circ$ and $l_{\text{NCP}} = 122.93192^\circ$,

$$\sin \delta = \sin \delta_{\text{NGP}} \sin b + \cos \delta_{\text{NGP}} \cos b \cos(l_{\text{NCP}} - l) \quad (22)$$

$$\alpha = \alpha_{\text{NGP}} + \text{atan2}(\cos b \sin(l_{\text{NCP}} - l), \cos \delta_{\text{NGP}} \sin b - \sin \delta_{\text{NGP}} \cos b \cos(l_{\text{NCP}} - l)) \quad (23)$$

with the inverse used for the l , b published in `/status`.

6.6 Solar position

Following Meeus [1], with T in Julian centuries from J2000.0:

$$L_0 = 280.46646 + 36000.76983 T \quad (\text{mean longitude}) \quad (24)$$

$$M = 357.52911 + 35999.05029 T \quad (\text{mean anomaly}) \quad (25)$$

$$C = 1.914602 \sin M + 0.019993 \sin 2M + 0.000289 \sin 3M \quad (\text{equation of centre}) \quad (26)$$

$$\lambda_\odot = L_0 + C \quad (\text{true longitude}) \quad (27)$$

followed by conversion from ecliptic to equatorial coordinates and precession from the equinox of date back to J2000. Accuracy is about 1 arcminute — entirely adequate against a 3° beam, and the reason the Sun is a usable pointing calibrator.

6.7 Lunar position

A truncated ELP/MPP02: mean elements L' , D , M , M' , F , the principal periodic terms in longitude and latitude, geocentric ecliptic to equatorial, then precession to J2000. Accuracy 5–10 arcminutes, comfortably inside the 0.5° encoder quantum.

6.8 Accuracy of the transforms

Compared against ERFA/astropy the transforms above agree to better than 0.5 arcminute RMS in ordinary observing conditions. Residual error is dominated by, in order: the encoder quantum (0.5°), the mount pointing model residual (Section 7), clock error, and only then the transforms themselves. Azimuth becomes ill-conditioned near the zenith, where it is also physically degenerate.

The coordinate and pointing code is exercised on the host by the `native` PlatformIO environment, which compiles `coordinates.cpp` and `pointing.cpp` exactly as they are flashed — a small `test/shim/` supplies the little of `Arduino.h` and `Preferences.h` that `pointing.cpp` touches — rather than testing a copy.

7 Pointing model and calibration

Everything in Section 6 produces a position on the sky. This section is about the difference between that and where the mount must actually be driven to see it. The transform lives in `esp32_controller_arduino/src/pointing.{h,cpp}`; the measurement of its parameters lives in `receiver_scheduler/sun_scan.py`.

7.1 Atmospheric refraction

Refraction is applied whether or not a pointing model is loaded, because it is physics rather than calibration. The controller uses Bennett’s formula [4] scaled for radio wavelengths:

$$R(h) = \frac{1.15}{60 \tan\left(h + \frac{7.31}{h + 4.4}\right)} \text{ degrees}, \quad (28)$$

with h the geometric altitude in degrees. The factor 1.15 accounts for radio refractivity being about 315 N-units against 273 optical, the difference being the water-vapour wet term. A nominal atmosphere is assumed: there is no weather station, and the dry/wet swing is well below the residual of a single calibration scan. The altitude is clipped to $[-1^\circ, 90^\circ]$, and the result forced to zero past the zenith where the tangent’s argument exceeds 90° and turns negative.

Equation (28) is duplicated as `refraction_deg()` in `sun_scan.py` and the two must stay identical. The controller adds refraction to every sky position it converts, so a scan’s measured error contains it; the fit removes it before solving, and if it did not, the fitted terms would absorb refraction and the controller would then apply it a second time. At 11° elevation that double count is about 0.09° — the same size as the effect the whole calibration is chasing.

7.2 The model

The mount error is described by the standard alt-az pointing terms [5]. Writing h for altitude and A for azimuth, the correction to be *added* to a true position to obtain the drive position is

$$\Delta h = \text{IE} + \text{AN} \cos A + \text{AE} \sin A - \text{TF} \cos h, \quad (29)$$

$$\Delta A = \text{IA} + (\text{AN} \sin A - \text{AE} \cos A) \tan h + \text{CA} \sec h + \text{NPAE} \tan h. \quad (30)$$

Table 14: Pointing model terms

Term	Name	Physical meaning
IE	Elevation index error	Zero-point error of the altitude encoder
IA	Azimuth index error	Zero-point error of the azimuth encoder
AN	Azimuth axis tilt, north	Rotation axis tipped towards north; $\text{AN} \approx \delta\phi$
AE	Azimuth axis tilt, east	Rotation axis tipped towards east; $\text{AE} \approx \delta\lambda \cos \phi$
CA	Collimation error	Beam not perpendicular to the altitude axis
NPAE	Axis non-perpendicularity	Altitude axis not perpendicular to the azimuth axis
TF	Tube flexure	Gravitational droop of the structure

The azimuth signs are opposite to the altitude ones for a reason worth remembering: tipping the azimuth axis towards north raises a source in the north but swings a source in the *east*

clockwise. The design matrix used by the fit has been checked against numerically rotated horizon coordinates rather than derived on paper alone.

All seven slots always exist and default to zero, so the four-term model actually fitted from Sun data is simply one with CA, NPAE and TF left at zero. A later, richer fit needs no change to the storage, the transform or the wire format.

7.3 The forward and inverse transforms

`trueToDrive()` applies refraction first and evaluates the model at the *apparent* position:

$$h_{\text{app}} = h_{\text{true}} + R(h_{\text{true}}) \quad (31)$$

$$h_{\text{drive}} = h_{\text{app}} + \Delta h(h_{\text{app}}, A_{\text{true}}) \quad (32)$$

$$A_{\text{drive}} = A_{\text{true}} + \Delta A(h_{\text{app}}, A_{\text{true}}) \pmod{360^\circ} \quad (33)$$

`driveToTrue()` inverts this by fixed-point iteration *on the forward transform itself*, so that whatever the forward direction does, the inverse undoes exactly that and the two cannot drift apart. Starting from $(h^{(0)}, A^{(0)}) = (h_{\text{drive}}, A_{\text{drive}})$ and writing F for `trueToDrive`,

$$(h^{(k+1)}, A^{(k+1)}) = (h^{(k)}, A^{(k)}) + \left[(h_{\text{drive}}, A_{\text{drive}}) - F(h^{(k)}, A^{(k)}) \right], \quad (34)$$

with the azimuth residual taken the short way round modulo 360° so that a guess either side of due north does not chase itself through a full turn. Three iterations are used.

Simply negating the correction evaluated at the drive position is only first-order accurate. That is adequate for the four-term model, but with CA and NPAE present the azimuth correction carries $\sec h$ and $\tan h$, and near the zenith the first-order answer is wrong by about a quarter of a degree — half the encoder quantum, which is not a rounding error. Three passes bring the error to well under a thousandth of a degree anywhere the mount can point.

7.4 Guards

Two clamps protect the mount from a broken or mis-scaled model. The altitude fed to the $\tan h$ and $\sec h$ terms is limited to 89° — at that altitude $\tan h$ is already 57, far past anything a real term needs, and azimuth is degenerate there anyway. The total correction on each axis is then limited to $\pm 10^\circ$; the whole measured error on this mount is around two degrees, so anything beyond that is not a pointing term. Clamping is silent by design, because the model is still reported by `/pointing` and a model that hits the clamp is therefore visible without a slew having to be commanded from it.

7.5 Where the calibration lives

The model is held in the controller’s own NVS namespace, separate from the settings namespace on purpose: the calibration is a measurement, not a preference, and “reset settings to defaults” must not silently discard it. An absent or malformed record leaves the identity transform — never a guess.

This replaced an earlier scheme in which the model was smuggled to the controller as a fictitious observer latitude and longitude plus a write to the operator’s pointing-offset boxes. That had two failure modes. The web UI reported a location the telescope is not at, and the offset half lived in RAM only, so every reboot silently discarded half the model: measured on 2026-08-19, the azimuth error that reappeared after a power cycle was 1.97° against a stored offset of 1.98° , about a quarter of the signal, lost without a word. The observer position is now the true site position and `/pointing/apply` does not touch it.

7.6 Measuring the model: the Sun scan

A single scan is an $n \times n$ raster (default 5×5 at 1.5° spacing) centred on the computed position of the Sun, measuring broadband power at each point.

7.6.1 Raster geometry

Offsets are specified on the sky. An altitude offset is applied directly; a cross-elevation offset Δx must be expanded into mount azimuth by the usual $\cos h$ factor:

$$h_{\text{cmd}} = h_{\odot} + \Delta h, \quad A_{\text{cmd}} = A_{\odot} + \frac{\Delta x}{\cos h_{\odot}}, \quad (35)$$

clamped to the usable azimuth range, with the Sun's position recomputed before each point so the raster does not smear as the Sun moves through it. Scans are refused within about 0.6° of the zenith, where $\cos h_{\odot}$ makes the expansion meaningless.

7.6.2 Beam fit

A circular two-dimensional Gaussian is fitted to the measured power over the offset grid,

$$P(x, y) = a \exp\left(-\frac{(x - x_0)^2 + (y - y_0)^2}{2\sigma^2}\right) + c, \quad (36)$$

whose centroid (x_0, y_0) is the pointing error of that scan and whose width gives the beam

$$\text{FWHM} = 2\sqrt{2 \ln 2} \sigma \approx 2.3548 \sigma. \quad (37)$$

The covariance returned by the fit supplies the per-scan centroid uncertainties, typically around 0.04° , which become the weights in the model fit below.

7.6.3 The datum: an absolute (T, D) pair

Each scan point records the Sun's true position T at that moment and the *reported drive position* D the mount actually settled on. The scan datum is their difference:

$$\Delta h = h_D - h_T, \quad \Delta x = [(A_D - A_T + 180^\circ) \bmod 360^\circ - 180^\circ] \cos h_T. \quad (38)$$

This pair is model-free. Whatever pointing model happened to be loaded during the scan only decided where the raster was aimed, and cancels out of the difference. That is what makes every fit *absolute* rather than a delta composed onto whatever came before, and it removes an entire class of composition error.

It also removes a noise floor. Because D is the position the Due rounded to and actually drove to, rather than the float the scheduler requested, the encoder grid is part of the measurement rather than an unmodelled 0.5° uncertainty sitting on top of it; the old quantisation term of 0.144° added in quadrature to every scan sigma is gone.

Scan records carry `record_version` (currently 2). Version 1 records were residuals against a model that is no longer knowable, and are skipped by the fit with a log message rather than silently pooled.

7.7 Fitting the model

7.7.1 Design matrix

For N scans the four fitted parameters are $\mathbf{x} = [\text{IE}, \text{IA}, \text{AN}, \text{AE}]^\top$ and the design matrix $\mathbf{M}$ is $2N \times 4$: rows $0 \dots N - 1$ are the altitude equations (29) and rows $N \dots 2N - 1$ the azimuth equations (30), i.e.

$$\mathbf{M} = \begin{pmatrix} 1 & 0 & \cos A_i & \sin A_i \\ 0 & 1 & \sin A_i \tan h_i & -\cos A_i \tan h_i \end{pmatrix}, \quad i = 1 \dots N, \quad (39)$$

stacked altitude block above azimuth block. The observation vector is

$$\mathbf{b} = [\Delta h_1 - R(h_1), \dots, \Delta h_N - R(h_N), \Delta x_1, \dots, \Delta x_N]^\top, \quad (40)$$

with refraction subtracted from the altitude half only — refraction is vertical to first order, so the cross-elevation errors are untouched.

7.7.2 Weighted solution

With σ_j the per-scan centroid uncertainty for observation j and $\mathbf{W} = \text{diag}(1/\sigma_j)$, the fit minimises

$$\chi^2 = \|\mathbf{W}(\mathbf{M}\mathbf{x} - \mathbf{b})\|^2 \quad (41)$$

by least squares, giving residuals $\mathbf{r} = \mathbf{b} - \mathbf{M}\mathbf{x}$, per-axis RMS residuals, and

$$\chi_\nu^2 = \frac{1}{2N-4} \sum_j \left(\frac{r_j}{\sigma_j} \right)^2. \quad (42)$$

The parameter covariance is

$$\mathbf{C} = (\mathbf{M}^\top \mathbf{W}^2 \mathbf{M})^+ \max(\chi_\nu^2, 1), \quad (43)$$

and the significance of each parameter is $|x_i|/\sqrt{C_{ii}}$. The floor of 1 on the chi-squared scale factor is deliberate: mount quantisation error is partly systematic, so mutually consistent scans give $\chi_\nu^2 \ll 1$, and scaling the covariance by it would shrink the parameter errors and overstate the very tilt significance that gates a weak calibration.

7.7.3 Geometry and quality gates

A fit is refused, with a message saying what to collect instead, unless all of the following hold. Azimuth coverage is measured as 360° less the largest gap in the sorted scan azimuths, which is the quantity that actually determines whether $\cos A$ and $\sin A$ are separable.

Table 15: Calibration acceptance criteria

Quantity	Requirement	Why
Valid scans N	≥ 4	Four parameters need two scans in principle; four spread scans are needed to detect poor geometry and give meaningful uncertainties
Azimuth coverage	$\geq 30^\circ$	Below this the two tilt terms are degenerate with the index errors
Condition number	$\leq 10^4$	Direct test that all four parameters are separable
Sun altitude	$0^\circ \leq h_\odot < 89.5^\circ$	$\tan h$ diverges at the zenith
Min. tilt significance	$\geq 3\sigma$	Geometry only says the parameters are separable, not that a tilt was measured
Reduced χ^2	≤ 4	Residuals far above the scan uncertainties mean something other than mount tilt is moving the pointing

The last two are checked again at *apply* time, not just at fit time, and a saved model that predates those checks is rejected rather than pushed.

7.7.4 The model in force

The four-term description above is the minimum the fit accepts. The model applied on 23 August 2026 carries six terms — IE -0.508 , IA $+1.685$, AN -0.464 , AE -0.902 , CA -1.044 and an azimuth scale of 0.000673 — fitted to 49 version-2 scans over three days. Its quality is judged by *prediction*, not by the size of any term: a blind fit to two days’ scans predicts the third day’s to 0.027° , and the pointing parameters are not measurements of anything mechanical — judge a model by leave-one-out prediction, never by whether an individual term looks plausible.

7.8 Drift scans on the drive grid

The mount parks on a grid: the Due rounds every target to the nearest encoder pulse, 0.5° in each axis, *after* the controller has applied refraction and the model. So “park where the source will be at T ” in fact parks up to 0.25° per axis away — a crossing up to a couple of minutes off T plus a cross-drift miss of up to 0.35° , which is what the first comparisons of Cas A, Crab and Sun drift scans against the planned T were partly seeing.

Since 26 August the scheduler reads the controller’s terms from `/pointing`, reproduces `trueToDrive` in Python (`drift_park.py`, validated in its tests against (drive, true) pairs the controller itself reported, to better than 0.006°), samples the source’s track ± 4 minutes about T , and parks on the grid point the track passes *closest* to, sending `/direct` the true position of that point so the controller’s own rounding lands on it exactly. The crossing time is then known in advance, to the second, and the miss is cross-drift only; both are written into the file (`drift_crossing_time`, `drift_crossing_offset_deg`) and the live plot marks the recorded crossing rather than the slot’s mid-point. After the slew the crossing is recomputed, model-free, from the true position the controller reports for the parked mount, and a drive position other than the planned one is logged — so the file never depends on the Python copy agreeing with the firmware. One thing learned writing it: the controller’s three-pass fixed-point inverse leaves 0.1° of azimuth at altitude 85° , where the *sec h* term is clamped; the copy here iterates to convergence, because the *forward* transform of what is sent must land on the grid point.

The first run was the Sun, parked on drive $39.5^\circ/218.0^\circ$ with the crossing predicted for 15:14:54 local: a Gaussian fit to the 355 records peaked $+0.11 \pm 0.01$ minutes later — 7 seconds, 0.028° along the drift — with a 5.10° FWHM on the sky. Two hours earlier, before the homing described next, the same scan had crossed 2.5 minutes late.

7.9 Homing first, and the encoder count check

The encoder counts are only as good as the last homing, and they can be lost. On 21 August 2026 the altitude axis lost counts during a horizon scan that moved it thousands of times: the sky level stayed rock steady while the *ground* signal faded across an azimuth sweep, a fixed tower dropped from a 0.0065 excess to 0.0009 , and an azimuth measured three hours earlier disagreed with itself. The dish was physically nowhere near where the controller believed, having driven past 90° altitude, which the drives permit. Nothing in software reported it; the tell was the baseline staying constant while the antenna contribution collapsed, which separates a pointing fault from a signal-path fault, since a lossy cable would drag both.

So a schedule entry may ask for a *physical homing before the observation* (Section 10). It re-references both axes against their limit switches and, while it runs, the scheduler polls the controller’s serial log — a 30-line ring buffer, about fifteen seconds — and reads the Due’s last reported position before each “limit reached”. The first approach’s reading is the count error accumulated since the previous homing (the stop is the true zero); the re-approach’s is the

repeatability. Both go in the log, and the first-approach values into the recording. A homing that faults or times out aborts the observation. The belt-and-braces version, in which the Due prints its counter at the stall so nothing depends on the 15-second buffer, is issue #24.

7.10 Wire format and application

The scheduler builds the document with `pointing_model_document()` and POSTs it to `/pointing/apply`. It deliberately carries the fitted terms and nothing derived from them:

```

1 {
2   "version": 1,
3   "fitted_utc": "2026-08-19T13:41:02Z",
4   "n_scans": 7,
5   "frame": "cross_elevation",
6   "terms": {"IE": -0.7213, "IA": 1.9842, "AN": 0.3117, "AE": -0.1042},
7   "site": {"lat": 55.902426, "lon": -4.307865},
8   "residual_rms_deg": {"alt": 0.06, "xel": 0.08}
9 }
```

Listing 2: Pointing model document

Unknown term names are ignored, so a richer model can be pushed to older firmware without bricking pointing. A missing `version` or a malformed number rejects the *whole* document and leaves the current model untouched — a half-applied model is worse than none. On success the controller replaces the model under the cross-task lock, writes it to NVS outside the lock, and returns the stored model for confirmation.

`/pointing/clear` zeroes the model and erases the NVS keys. A cleared model and an all-zero model behave identically — both are the identity transform — so it is safe to use after mechanical work without leaving pointing in a special state.

7.11 Operating procedure: a calibration day

1. Ensure the receiver is free: a Sun scan and a calibration day both claim the SDR and will refuse to start if an observation or a manually booted receiver holds it.
2. Start a calibration day from the scheduler (POST `/api/calday/start`) with a scan interval, typically 30 minutes. It waits for sunrise if necessary, runs a scan every interval while the Sun is above 5° , and finishes at sunset.
3. Each hardware scan re-homes the mount first, so every datum is referenced to a freshly measured physical limit. A rejected scan is re-homed and retried once; only the final outcome counts as a failure.
4. Run for a full day, or at least a morning *and* an afternoon set. This is what produces the azimuth spread the tilt terms need; an afternoon-only set will fit and then be refused at the significance gate.
5. Fit with POST `/api/calday/fit`. This writes `pointing_model.json` and a four-panel `calibration_day.png` showing measured against modelled errors in both axes, the residuals, and a summary.
6. Apply with POST `/api/calday/apply`. The gates of Table 15 are enforced here before anything reaches the telescope.
7. Verify: GET `/pointing` on the controller should return the same terms, and they must survive a power cycle.

After mechanical work on the mount, clear the accumulated scan data (`/api/calday/clear`) as well as the model — old scans describe a mount that no longer exists.

8 Observatory host and the controller link

8.1 What the link is

The observatory computer carries a **second Ethernet card**. The controller hangs off it on a private point-to-point link that the host owns completely: it serves the address, resolves names, and routes the link out to the internet. The first card stays on the site LAN and is how people reach the computer.

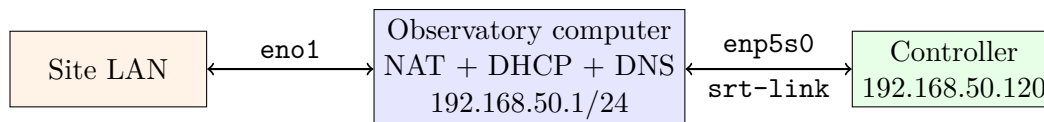


Figure 3: The private controller link. Interface names are illustrative; the card’s bus position fixes them.

8.2 Why

On the site LAN the controller had whatever address the site handed it, and that address was hard-coded in seven places in the code and twenty-five in the documentation. Every renumbering was a code change. On a private link the address is permanently ours.

No firmware change is needed for any of this. The controller stays on DHCP and its address is pinned by MAC on the host, so nothing on the controller is configured for this link and the whole arrangement reverses by moving one cable back to the site switch. *That reversibility is the point* and should be preserved by anything built on top of it.

The one thing the link breaks, if built carelessly, is time — and by the argument of Section 5 a clock error is a pointing error, arriving overnight and tedious to diagnose after the fact. Routing the link out is therefore not optional.

8.3 Address plan

The subnet must avoid the site LAN, 192.168.4.0/24 (the controller’s own WiFi AP) and 192.168.1.0/24 (common home-router range). 192.168.50.0/24 is used. Nothing depends on the numbers, but changing them means changing `DEFAULT_ETH_*` in the controller’s `config.h` too.

Table 16: Address plan

Host	Address
Observatory computer, second card	192.168.50.1
Controller	192.168.50.120 (DHCP, pinned by MAC)

8.4 Hardware

A TP-Link TG-3468: PCIe x1, Realtek RTL8168, driven by the in-kernel `r8169` module, so Ubuntu brings it up with no driver install.

Avoid cheap “Intel I210” listings. A genuine i210-T1 is the better card in the abstract, but real ones are considerably more expensive than the marketplace listings, and those listings are a well-documented counterfeit market. Counterfeits frequently carry cloned or duplicated MAC addresses — exactly the failure this design depends on not happening, since the controller’s address is pinned by MAC. The link carries a status poll and a coordinate stream and the controller’s PHY is 100BASE-TX anyway: buy on driver reliability, not speed.

8.5 Building the link

Do this while the controller is still on the site switch; nothing is at risk until the cable moves.

```

1 sudo nmcli connection add type ethernet \
2   con-name srt-link \
3   ifname enp5s0 \
4   ipv4.method shared \
5   ipv4.addresses 192.168.50.1/24 \
6   ipv4.never-default yes \
7   ipv6.method ignore

```

Listing 3: Create the shared connection

ipv4.method shared

The important one. It sets the address, enables IP forwarding, installs the NAT rules, and runs a dnsmasq on the interface providing both DHCP and a DNS forwarder — all four in one setting.

ipv4.never-default yes

Stops the host routing general traffic down a link with only a microcontroller on the far end.

ipv6.method ignore

Nothing here speaks IPv6.

The controller's address is then pinned by MAC so it can stay on DHCP:

```

1 dhcp-host=70:4B:CA:58:59:8B,192.168.50.120,srt-controller

```

Listing 4: /etc/NetworkManager/dnsmasq-shared.d/srt.conf

Shared mode leases from .10 upward and a `dhcp-host` entry takes precedence within that range, so .120 is safe. Activation without a cable attached succeeds; the interface stays `NO-CARRIER` until the controller is moved across, which is normal. Verify with `sysctl net.ipv4.ip_forward` (must be 1), `nft list ruleset | grep -i masquerade`, and `pgrep -af "dnsmasq.*enp5s0"`.

8.6 Firewall

Note first that `systemctl is-active ufw` reports “active” even when the firewall is not enforcing. `sudo ufw status` is the real answer.

```

1 sudo ufw limit 22/tcp comment 'ssh (rate limited)'
2 sudo ufw allow in on enp5s0 comment 'SRT private link'
3 sudo ufw route allow in on enp5s0 out on eno1 comment 'SRT link NAT out'
4
5 sudo ufw default deny incoming
6 sudo ufw default allow outgoing
7 sudo ufw default deny routed
8 sudo ufw enable

```

Listing 5: Firewall rules

Two of those are non-obvious and both were learned the hard way.

- **route allow ...out on eno1 is what keeps the controller's clock working.** The default forward policy is deny, which silently blocks the NAT. The controller then never syncs time again and the mount mispoints overnight.

- **allow in on enp5s0 is what keeps OTA firmware updates working.** `espot` sends its invitation to the controller and then expects the controller to connect *back* to the host on an ephemeral port — a different one every run — so it cannot be allowed by port number. Without this rule the upload fails with the unhelpful `No response from device`, because the invitation succeeds and only the callback is blocked. Trusting the link interface as a whole is reasonable: it carries exactly one device, on a subnet this host owns and serves.

Nothing needs an inbound rule on the site interface. In particular do not open port 5000 — see Section 2.

8.7 Verification

```

1 ping -c3 192.168.50.120
2 curl http://192.168.50.120/ping           # -> ok
3 curl http://192.168.50.120/network      # eth_ip and the DNS diagnostics
4 curl http://192.168.50.120/status       # live drive and true alt/az
5 getent hosts srt-controller.local       # mDNS, needs avahi on the host

```

Listing 6: Basic checks from the observatory computer

Then the check that is easy to get wrong. `/time/status` reporting `"source":"NTP"` **does not prove the link routes**. The firmware carries a numeric NTP fallback behind the pool name, so the clock syncs whether or not DNS works, and will report NTP on a host that cannot resolve anything at all — a true statement about the clock and a useless one about the network.

What proves it is a *fresh sync after a reboot*. Reboot the controller (an OTA upload will do), then query `/time/status:sync_count` must have reset to 1, confirming a real reboot rather than a stale reading, with a small `last_sync_age_s`. That combination means SNTP packets left the host, reached a time server and came back.

DNS itself can be confirmed either from `/network` (Section 5) or by temporarily adding `log-queries` to the `dnsmasq` drop-in and watching for `query[A] pool.ntp.org` from 192.168.50.120 in the NetworkManager journal.

Finish by running one OTA upload deliberately, since that is the operation most dependent on the firewall being right:

```

1 cd esp32_controller_arduino
2 ~/.platformio/penv/bin/pio run -e wt32-eth01-ota -t upload

```

8.8 Recovery

The controller has independent safety nets, so a wrong address is not a bricking risk:

1. Move the cable back to the site switch. Because the controller was never reconfigured, this restores the previous state exactly.
2. The WiFi AP stays active — SSID `SRT_Controller`, UI at `http://192.168.4.1` — regardless of the Ethernet settings.
3. Invalid static configuration falls back to DHCP, so a malformed address leaves the controller reachable rather than dark.
4. Last resort is the FT232-and-buttons serial flash at the telescope.

8.9 What the link deliberately does not do

Access to the controller from other machines on the site network is *not* set up. The host NATs outbound for the controller; nothing routes inbound. Providing it would need a forwarding rule or reverse proxy, which partly re-exposes the thing the private link exists to protect. Remote use at Acre Road is `waypipe` into the observatory computer instead.

The site interface keeps whatever the site gives it. Nothing here depends on its address: the masquerade rule matches on the link subnet, the firewall rules name interfaces rather than addresses, and `never-default` keeps the link out of the routing decision.

The full build procedure, with the commands in order and the checks between them, is in `docs/OBSERVATORY_HOST_SETUP.md`.

9 Beam, radiometry and velocity frames

Everything so far has been about putting the beam somewhere. This section is about the beam itself, about what the numbers coming out of the receiver mean, and about the one correction the system deliberately does *not* apply.

9.1 Beam and aperture

The dish is 3.0 m across and the observing wavelength is

$$\lambda = \frac{c}{\nu_{\text{HI}}} = \frac{2.998 \times 10^8}{1.4204 \times 10^9} = 0.2111 \text{ m.} \quad (44)$$

For a uniformly illuminated circular aperture the half-power beamwidth is

$$\theta_{\text{FWHM}} \simeq 1.22 \frac{\lambda}{D} = 1.22 \times \frac{0.2111}{3.0} = 0.0858 \text{ rad} = 4.9^\circ, \quad (45)$$

and a realistic illumination taper widens this slightly. This single number sets the scale of nearly every tolerance in the preceding sections:

- The 0.5° encoder quantum is about one tenth of a beamwidth, so pointing quantisation costs almost nothing in signal.
- The residual pointing error after calibration, a few hundredths of a degree, is negligible against it — but the *uncalibrated* error of about 2° is nearly half a beamwidth, which is why the calibration of Section 7 is worth the trouble.
- The Sun-scan raster spacing of 1.5° is roughly 0.3 beamwidths, and a 5×5 grid spans $\pm 3^\circ$, comfortably over the half-power points in both axes — which is what the Gaussian fit needs to find a centroid.

The corresponding beam solid angle for a Gaussian beam is $\Omega_b \simeq 1.133 \theta_{\text{FWHM}}^2 \approx 27$ square degrees. An unresolved source of solid angle Ω_s and brightness temperature T_b is therefore diluted to an antenna temperature

$$T_A \simeq \eta_{\text{mb}} T_b \frac{\Omega_s}{\Omega_b}, \quad (46)$$

with $\eta_{\text{mb}} \approx 0.7$ a typical main-beam efficiency for a dish of this kind. Equation (46) is why the Sun is usable as a pointing calibrator: its 0.53° disc is only $\Omega_s/\Omega_b \approx 0.008$ of the beam, but its 21 cm brightness temperature is of order 10^5 K, so it still arrives as an antenna temperature of order 10^3 K — far above everything else in the sky and unmistakable in a raster. That is the figure the receiver’s on-Sun power-bar ceiling (`CAL_BAR_SUN_K`) is set from.

9.1.1 The measured beam, and how well it is known

The $1.22\lambda/D = 4.9^\circ$ above is the aperture ideal. The value the code carries — in `instrument.py`, shipped to both simulators and the scheduler so nothing can disagree about it — is the measured 5.164° from the Sun-scan rasters. On 26 August 2026 three independent measurements put the beam *narrower* than that: a Sun drift scan gave 4.64° along the drift; the Cassiopeia A drift’s amplitude and width wanted $\sim 3.8^\circ$; and scanning the beam as a free parameter in the gain fit of Section 9.4.3 minimised the residual near 4.0° on two plane fields. The Sun rasters’ points sit on the 0.5° drive grid, which broadens a fitted raster width in a way a continuous drift does not, which may be the discrepancy.

It matters because the beam is degenerate with the gain in exactly the damaging direction: a wider beam dilutes the model’s H I peak and the fit compensates with gain, so moving the assumed beam from 4.0° to 5.16° shifts the fitted gain by 6% and T_{sys} by 22 K, at a correlation of 0.995 either way. The assumed beam is thus the largest silent systematic in the temperature scale. Settling it needs a measurement independent of the H I model and the drive grid — continuous drift scans of Cyg A and Tau A at their transit elevations and the Sun at several — after which the gain is refit and the Cas A excess re-examined. Until then the value stands and the uncertainty is stated.

9.2 System temperature

The receiver’s default blank-sky system temperature is 70 K, built up as follows.

Table 17: Blank-sky system temperature budget (DEFAULT_CAL_TEMP_K)

Contribution	Temperature	Source
Low-noise amplifier	43 K	SAWbird+ H1, noise figure ≈ 0.6 dB
Sky	10 K	CMB, atmosphere, galactic background
Spillover and feed losses	17 K	Ground pickup past the dish edge
Total	70 K	

The amplifier figure follows from the noise figure in the usual way,

$$T_{\text{rx}} = T_0 \left(10^{F/10} - 1 \right) = 290 \left(10^{0.06} - 1 \right) = 43 \text{ K}, \quad (47)$$

with $T_0 = 290$ K the reference temperature. The remaining two terms are estimates rather than measurements; a measured T_{sys} is what the calibration below is for.

9.3 The radiometer equation

The RMS fluctuation in a single spectral channel of width $\Delta\nu$ after an integration τ is

$$\Delta T = \frac{T_{\text{sys}} + T_{\text{A}}}{\sqrt{n_{\text{pol}} \Delta\nu \tau}}, \quad (48)$$

where n_{pol} is the number of polarisations summed (one, for this receiver). The signal appears in the numerator as well as the system temperature because the source is itself noise: a channel sitting on a bright H I line fluctuates more than a channel beside it, and the simulator of Section 11 models exactly this.

As a worked example, at the default settings — 2.4 MHz of bandwidth over 4096 channels, so $\Delta\nu = 586$ Hz — and taking $T_{\text{sys}} = 200$ K on a modest line,

$$\Delta T = \frac{200}{\sqrt{586 \times 60}} = 1.1 \text{ K per channel in one minute.} \quad (49)$$

Galactic H I at low latitude reaches tens of kelvin, so the line is visible in a single minute; a high-latitude profile of a few kelvin needs tens of minutes, and Equation (48) is how that observation is planned.

9.3.1 Total-power instability

Equation (48) is the ideal. The practical form carries a second term for gain fluctuation,

$$\frac{\Delta T}{T_{\text{sys}}} = \sqrt{\frac{1}{\Delta \nu \tau} + \left(\frac{\Delta G}{G}\right)^2}, \quad (50)$$

and it is why Dicke switching exists. It is normally absent from H I tooling because line work is either per-channel, where the thermal term dominates, or switched, where the common mode cancels. An unswitched *band-integrated* drift scan is the one regime where neither escape applies: the thermal term divides by the number of channels while the gain term does not, so at $\Delta \nu \tau \approx 2 \times 10^8$ the common-mode wobble sits well above the band-mean thermal floor even while the per-channel scatter is still thermal. Measured on a settled run — second differences of the band mean over 99 records, stationary across halves — it is $\Delta G/G = 2.3 \times 10^{-4}$, or 0.079 K per 60 s record. Both simulators add it in quadrature to drift-scan samples only; in a spectrum it is a flat common-mode offset seven times under the per-channel thermal noise and removed exactly by any baseline handling, so modelling it there would add noise the science cannot see. It was first installed ten times too large, from first differences on a freshly started receiver, where a warm-up transient and real sky drift were read as instability; second differences on a settled run are the measurement. The practical corollary is an Allan minimum for band power of about 5 s at this bandwidth — beyond that, more samples beat longer samples — and the upgrade path, if band-integrated continuum ever becomes a science goal, is Dicke-style switching of the existing noise diode.

9.4 Temperature calibration

Two calibration paths exist, and only one of them is currently automated.

9.4.1 Single-point blank-sky calibration

The receiver GUI offers a one-click calibration: with the dish on blank sky, the mean total power over the sliding window is declared to be the entered temperature, giving a scale factor

$$k = \frac{T_{\text{cal}}}{\bar{P}} \quad [\text{K per arbitrary unit}], \quad (51)$$

after which the total-power chart reads in kelvin and the power bar is re-spanned from just below blank sky (50 K) to the approximate on-Sun level. The scale is invalidated automatically if the bandwidth or FFT size changes, because those alter the raw power scale. It is a one-point calibration against an *assumed* T_{sys} , so it fixes the units, not the accuracy. Section 17.8.3 shows how a blank-sky and Sun pair measures the number that is currently typed into that box.

9.4.2 Noise diode and the Y-factor

The calibrator is a noise diode on GPIO 26 of the Due, switchable from the web UI, the API and the schedule, and its state is recorded both in the HDF5 `calibrator` attribute and in a `_cal` filename suffix. At present the system *records* the diode state and does not derive a gain solution from it. The intended use, when a diode of known excess temperature T_{cal} is characterised, is the standard Y-factor:

$$Y = \frac{P_{\text{on}}}{P_{\text{off}}}, \quad T_{\text{sys}} = \frac{T_{\text{cal}}}{Y - 1}, \quad T_{\text{A}} = T_{\text{sys}} \left(\frac{P_{\text{source}}}{P_{\text{off}}} - 1 \right). \quad (52)$$

Because the diode fires ahead of the low-noise amplifier, the same measurement tracks gain drift through the observation, which the blank-sky method cannot. Interleaving on and off spectra is therefore the upgrade path for absolute work; the metadata needed to do it offline is already being written to every file.

9.4.3 The automated gain fit against the survey

The calibration actually in force is fitted, on the RF calibration tab, in two measurements. First the **bandpass template**: the receiver points at empty sky — the Lockman Hole, $l = 150$, $b = +52$, the emptiest hydrogen field in the north, reached by the tab’s *Go to Lockman Hole* button — and a polynomial is fitted to the mean spectrum with the H I line masked and bridged, giving the instrument’s spectral shape. Since the fixed instrument there are two templates, one per product, the continuum one normalised over the H I band so that the single gain serves both; dividing a recording by its template flattens it without moving its level. The template is refused at any tuning it was not fitted at.

Then the **gain and system temperature**: the receiver records a plane field, the recorded spectrum is bandpass-corrected and binned onto the survey model’s grid, and a straight line

$$\text{counts}(\nu) = G (T_{\text{sys}} + T_{\text{A}}^{\text{model}}(\nu)) \quad (53)$$

is fitted by least squares, G the slope and T_{sys} the intercept over slope. $T_{\text{A}}^{\text{model}}$ is the simulator’s beam-weighted H I prediction for that direction and time, shifted into the recording’s topocentric frame. A bound holds T_{sys} above 50 K during the fit, not after: a fit driven onto that bound (as one was, behind an orphaned receiver recording demo noise) is against the bound, not a measurement, and is not a calibration. The last such gain in force, on $l = 72$, $b = 0$ (26 August 2026), was 1.197×10^{-5} counts K⁻¹ at $T_{\text{sys}} = 347$ K, correlation 0.998.

Two things are worth stating plainly about this scheme. It calibrates H I against an H I model, which is circular for line science and separates G from T_{sys} only through the line’s contrast — a field with a weak line gives a confident but wrong split. And it assumes the beam, which is the largest systematic in the scale (Section 9.1.1). The way past both — a receiver-generated pseudorandom comb, radiated from a vertex antenna and recovered by cross-correlation, giving gain, passband and T_{sys} per record with no sky model — is recorded as issue #26.

9.4.4 Fitting a drift scan: total power against the model curve

A drift scan is a total-power measurement, and it is fitted differently. As the parked beam drifts, the band-integrated power traces $G (T_{\text{sys}} + \bar{T}_{\text{model}}(t))$, with $\bar{T}_{\text{model}}(t)$ the simulator’s band-mean sky — the H I cube, the diffuse continuum and the bright sources, through the measured beam — at the fixed alt/az as it drifts. Two parameters, G and T_{sys} , follow by least squares. This needs *no* bandpass template: the band-integrated power is the spectrum summed over channels, and the bandpass shape is the same constant factor on every record, absorbed into G . So a drift scan at any tuning can be compared with the simulator, where the per-channel fit rightly refuses a tuning it has no template for. The two gains are not the same number — this one carries the band-mean of the bandpass shape — so a total-power fit is drawn and reported, never adopted as the per-channel calibration. It is measured over the **continuum band only**, the H I line excluded, so that the “constant factor” really is constant: the line sits on channels with a different gain, and a fit that included it came out a band-weighted mixture.

The same total power feeds a **live trace** while a drift scan or solar-flux track runs, read from the text sidecar so the HDF5 need not be reopened per record. Its time axis is the observation’s own window, fixed from the start with the beam-crossing time marked, so the plot fills in towards

a known crossing rather than autoscaling; a tracked spectrum gets no such trace, because its band power is meant to be flat and an autoscaled picture of it is a picture of the noise. The solar-flux monitor turns the band median into flux above the atmosphere via the antenna theorem and the measured beam, and read 112 SFU on 25 August against RSTN’s 88 (Learmonth) and 108 (San Vito) — agreement within the reference network’s own spread, and the first end-to-end check of bandpass, gain, T_{sys} , beam and the antenna theorem against an external standard.

9.5 Velocity frames — what is recorded, and what the plots draw

The receiver’s spectrum carries a second axis in velocity, computed in the radio convention

$$v = c \frac{\nu_0 - \nu}{\nu_0}, \quad \nu_0 = 1420.405752 \text{ MHz}, \quad (54)$$

equivalently $1 \text{ km s}^{-1} \equiv 4.74 \text{ kHz}$ at this frequency.

The recorded axis is topocentric. The file holds sky frequency exactly as observed, with no correction to the Local Standard of Rest, and that is deliberate: the receiver process does not know where the dish is looking, and a recording must re-reduce correctly whatever is later learned about the direction or the clock. **The plots, since 25 August 2026, draw an LSR axis** whenever the direction and epoch are known — which is every scheduled observation — and state in their header exactly what was applied, for example “*LSR frame — topocentric +14.00 km s⁻¹ (Earth’s orbit and rotation, Sun’s motion, towards $l = 76.2$, $b = +5.8$) and receiver clock +1.09 km s⁻¹ applied at the mid-time 17:56 UT; the file itself is recorded topocentric*”. The Observe tab’s details table carries the same sentence. A recording with no known direction (a console recording) keeps a topocentric axis and says so. The consequence of the frames must still be understood by anyone reducing the files themselves, because the offset is not small:

Table 18: Velocity terms between the topocentric and LSR frames

Term	Maximum	Period
Earth rotation	0.46 km s^{-1}	Daily
Earth orbital motion	29.8 km s^{-1}	Annual
Solar motion towards the LSR apex	20.0 km s^{-1}	Fixed per direction
Combined	$\sim 50 \text{ km s}^{-1}$	$\equiv 237 \text{ kHz}$

A quarter of a megahertz is a large fraction of a galactic H I profile, and the annual term reverses over six months, so uncorrected spectra of the same source taken in different seasons will not superpose.

The correction is applied in analysis. Writing v_{topo} for the axis as recorded, v_{bary} for the observer’s barycentric radial velocity towards the source at the time of observation, and $\hat{n}$ for the unit vector to the source,

$$v_{\text{LSR}} = v_{\text{topo}} + v_{\text{bary}} + \mathbf{U} \cdot \hat{n}, \quad (55)$$

where $\mathbf{U}$ is the kinematic solar motion, which defines LSRK as 20.0 km s^{-1} towards RA 18^{h} , Dec $+30^\circ$ (B1900). Every quantity needed is in the HDF5 file: the UTC timestamps, the target coordinates and the observer position. In practice v_{bary} comes from `astropy’s SkyCoord.radial_velocity_correction("barycentric", ...)`, and the apex projection is a single dot product. The reference implementation is `frame_offset()` in `astro_simulator/astro_simulator.py`, which converts between the LSR, barycentric and topocentric frames in exactly this way; the plots use it in one direction (`lsr_offset_km_s` in `observation_plot.py`) and the gain fit of Section 9.4.3 in the other, shifting the LSR-frame survey model into the recording’s topocentric frame before comparing.

The conversion is evaluated at the observation’s own mid-time and epoch, never “now”: the barycentric term moves by 1.95 km s^{-1} in a week. It was validated at the galactic anticentre, where rotation contributes nothing along the line of sight: the $l = 184$, $b = 0$ observation of 25 August 2026 peaked at -11.17 km s^{-1} topocentric and lands at $+3.75 \text{ km s}^{-1}$ corrected, against HI4PI’s $+3.97$ — agreement to 0.22 km s^{-1} , from 15 km s^{-1} before.

9.5.1 The receiver clock

A third term is the B210’s own reference oscillator. Its error scales the whole frequency axis, which across a few MHz is a pure velocity shift; the constrained gain fits (those whose H I line was strong enough to hold the shift, correlation ≥ 0.99) give $-2.36 \pm 0.27 \text{ ppm}$ over 18 hours — $\pm 0.08 \text{ km s}^{-1}$, well inside the $\pm 2 \text{ ppm}$ the part is specified at — and that stored value is applied to the plotted axis as well. The scatter of the unconstrained fits, -6.6 to $+18.4 \text{ ppm}$, tracks line strength and not time: a shift with no line to hold it slides onto whatever is nearby and reports a confident number. Hence `trustworthy_velocity_shift`: the clock is stable, and only a fit that could measure it is believed.

10 H I receiver and observation scheduler

10.1 Receiver overview

`b210_h1_receiver.py` is a GNU Radio spectrum analyser for 21 cm observations, offering real-time FFT with configurable integration, a live spectrum and waterfall display (PyQtGraph), and continuous logging to HDF5. It has two front ends on one flowgraph: the console window, for warm-up and bench work, and a *headless* recorder (`-headless`) that imports no Qt at all and is the only path a scheduled observation uses — the observatory is worked over `ssh`, and an observation must not depend on a display.

10.1.1 The fixed instrument

Since 26 August 2026 the receiver’s tuning is **not a per-observation choice**. `tuning.fixed_instrument()` defines it once — LO 1418.905752 MHz (the line minus 1.5 MHz), 8 Msps, 40 dB of gain — the scheduler hands it to every headless receiver as the `H1_INSTRUMENT` environment variable, and the `receiver_*` configuration keys override it deliberately, from the Configuration tab, which warns before saving a change: the bandpass templates and the gain calibration belong to the tuning, and every recording after a change is in counts until they are re-measured. From the one 8 MHz stream the recorder computes **two products**:

H I sub-band

1419.006–1422.306 MHz — topocentric -400 to $+300 \text{ km s}^{-1}$, the whole disc and every major high-velocity-cloud complex with the Earth’s $\pm 50 \text{ km s}^{-1}$ absorbed — cut from the stream by a frequency-translating decimator to 4 Msps and a 1024-point FFT: 3.91 kHz (0.82 km s^{-1}) channels, 845 of them inside the band. The narrowest galactic H I is $2\text{--}3 \text{ km s}^{-1}$ wide and the HI4PI model 1.29 km s^{-1} , so this Nyquist-samples anything a 5° beam can show.

Continuum product

the whole 8 MHz in 1024 channels of 7.8 kHz, of which the **continuum band** 1415.706–1418.806 MHz — 3.1 MHz with no hydrogen in it, on the good side of the SAW filter — is what every continuum measurement uses. Continuum *always* excludes the H I band.

The LO sits on the positive-velocity edge of the sub-band, where there is no emission beyond $+300 \text{ km s}^{-1}$, so its DC artefact can never touch a line and is outside the H I product altogether; it falls inside the continuum product and is masked there. The 8 Msps came from a sample-rate ladder run on the observatory host that day: 4.5, 8 and 10 Msps ran with no UHD overflows, 12 Msps overflowed steadily and 16 and 20 lost most of the stream — the host’s FFT thread, not USB, being the limit (a single thread of the i7-4790 does 60 Msps of 2048-point FFTs; the old flowgraph spent most of it elsewhere). M31, whose approaching half reaches -600 km s^{-1} , lies past the 8 Msps edge and remains a console-window case with free tuning.

Two consequences for anyone reading counts. The per-bin power is normalised by the transform length, so the counts scale — and the gain in counts per kelvin — does not depend on how many channels the band is cut into (the first two-product flowgraph dropped that normalisation and halving the channel count silently halved every count). And a headless receiver that cannot open the radio it was asked for **fails**, rather than falling back to the demo noise source as the window does: on 26 August a gain calibration recorded three minutes of Gaussian noise behind an orphaned receiver that still held the B210, and the fit took it for the sky.

10.1.2 Overflows

UHD reports a dropped block of samples by printing a bare 0; nothing reaches the flowgraph. The recorder therefore routes its own standard streams through a pipe, counts UHD’s “ N overflows occurred” messages, and writes the count per record into the file. At 16 Msps a file looked perfect — tidy 3s records — while most of the samples had been dropped; the count is what tells that file from a good one, and the Observe tab flags any record with one.

Table 19: Supported SDR platforms

SDR	Frequency range	Sample rate	Notes
Ettus B210	70 MHz – 6 GHz	up to 56 MHz	Recommended
RTL-SDR (R820T)	24 – 1766 MHz	up to 2.4 MHz	Budget option
Demo	—	—	Simulated data, no hardware

```

1 python b210_h1_receiver.py --headless --sdr b210      # as the scheduler runs it:
   the fixed instrument
2 python b210_h1_receiver.py --sdr b210 --gain 40 --sample-rate 2.4e6 # console
   window, free tuning
3 python b210_h1_receiver.py --sdr demo

```

Listing 7: Receiver command line

In headless mode `-sample-rate` and `-gain` are noted and ignored: the fixed instrument decides, so a stale launcher cannot retune a scheduled observation. Two receiver instances contend for the one B210, which is the usual cause of a receiver that exits immediately — and, since the fixed instrument, of one that refuses to start rather than silently recording demo noise.

10.2 Recordings: where they go and what they are called

Every recording lands in `data/observations/`, named `YYYYMMDD_HHMMSS_<mode>.h5` and nothing more. The name carries only what the file cannot: the time, which the name cannot be wrong about, and the mode — **track** if the mount followed the sky, **drift** if it was parked and the sky moved through the beam (an alt/az entry is a drift scan), **manual** for a console recording, where nobody commanded the mount. Target, comment and calibrator state are attributes, not filename parts: a second copy in the name was free to disagree with the first after any rename. `data/` itself stays the general working folder for plots, horizon partials and calibration fits; it

once held 248 files of which 11 were observations. A mid-run change of spectral geometry rolls to a fresh timestamped file with the reason recorded as `segment/segment_reason` attributes.

10.3 HDF5 output format

Table 20: HDF5 datasets. The H1 product keeps the names every recording has always had; the continuum product is beside it.

Dataset	Shape	Description
<code>frequency_hz</code>	$(N_{\text{ch}},)$	H1 product axis, sky frequency in Hz (845 channels)
<code>spectra_kelvin</code> <i>or</i> <code>spectra_linear</code> <code>bandpass_correction</code> , <code>bandpass_valid</code>	$(N_{\text{rec}}, N_{\text{ch}})$ $(N_{\text{ch}},)$	H1 product, one record per integration The per-channel correction applied at write time, and where the template speaks
<code>frequency_hz_wide</code>	$(N_{\text{wide}},)$	Continuum product axis (1024 channels over 8 MHz)
<code>spectra_wide_kelvin</code> <i>or</i> <code>spectra_wide_linear</code> <code>bandpass_correction_wide</code> , <code>bandpass_valid_wide</code>	$(N_{\text{rec}}, N_{\text{wide}})$ $(N_{\text{wide}},)$	Continuum product Its correction
<code>timestamps</code>	$(N_{\text{rec}},)$	Unix time at the end of each record
<code>integration_times</code>	$(N_{\text{rec}},)$	Actual integration per record
<code>overflows</code>	$(N_{\text{rec}},)$	UHD overflows during the record (Section 10.1.1)

The units are in the dataset name. `spectra_kelvin` means the bandpass template and gain in force applied to this tuning and were applied at write time; `spectra_linear` means they did not and the data are raw counts. An attribute (`spectra_units`) says the same, but the name is the guard: asking for the wrong one raises `KeyError` rather than silently returning the other scale. Writing in kelvin is safe only because it is exactly reversible — `bandpass_correction`, `applied_gain_counts_per_k` and `applied_t_sys_k` all travel in the file, and `observation_plot.read_observation` reverses it so the fitting code is always handed counts. That is not a wasted round trip: fitting a gain from spectra a gain has already been applied to is circular, and would return unity and a system temperature of zero while looking like a perfect calibration.

A recording can be read while it is being written. Files are opened with `libver='latest'` and switched to HDF5's single-writer/multiple-reader mode once every dataset and attribute exists — the constraints (nothing created afterwards, anything extended chunked) were already met by the layout. Readers go through `open_readonly`: a plain open, and on the lock the SWMR open, seeing the file as of the writer's last flush, which is per record. This is what lets the Observe tab plot and fit a live recording, and a notebook open one with `h5py.File(path, 'r', swmr=True)`. A one-line-per-record text sidecar, `<name>.live.jsonl`, still feeds the live band-power plots: one number per record from a text file is cheaper to poll than reopening an HDF5, and immune to a torn read. It carries the H1-band median, the continuum-band median and the overflow count.

Table 21: HDF5 root attributes

Attribute	Description
<code>sdr_type</code>	b210, rtl-sdr or demo
<code>center_freq_hz</code> , <code>sample_rate_hz</code> , <code>gain_db</code>	The LO, rate and gain — the
<code>instrument</code>	The whole fixed-instrument de
<code>h1_band_hz</code> , <code>continuum_band_hz</code>	The two bands, [low, high]
<code>dc_artefact_freq_hz</code>	Where the LO's DC artefact li
<code>spectra_units</code> , <code>spectra_wide_units</code>	K or counts, per product
<code>applied_gain_counts_per_k</code> , <code>applied_t_sys_k</code>	The calibration applied at wri
<code>bandpass_template</code> , <code>bandpass_template_wide</code> , <code>gain_calibration</code>	The calibration documents in as JSON, whether or not they away from the machine that m
<code>beam_fwhm_deg</code> , <code>effective_area_m2</code> , <code>site_lat_deg</code> , <code>site_lon_deg</code> , <code>site_height_m</code>	The measured beam and the s
<code>nominal_integration_time</code> , <code>created</code>	Target integration; ISO 8601 U
<code>segment</code> , <code>segment_reason</code>	Present on a file that continue

Table 22: HDF5 observation metadata (added when launched by the scheduler)

Attribute	Description
<code>obs_name</code> , <code>comment</code>	Name, and the free text typed into th (absent when none)
<code>observation_mode</code>	track, drift or manual — the word in
<code>coord_system</code>	altaz, radec, galactic, object, drift
<code>object_name</code>	Solar-system object if applicable
<code>coord1_deg/min/sec</code> , <code>coord2_deg/min/sec</code>	Target coordinates
<code>calibrator</code>	1 if the noise source was on
<code>duration_minutes</code> , <code>start_date</code> , <code>start_time</code>	The scheduled slot
<code>tle_text</code>	TLE data for satellite observations
<code>drift_frame</code> , <code>drift_window_min</code> , <code>drift_beam_time</code>	Drift scans: the source frame, the half planned crossing time T
<code>drift_alt</code> , <code>drift_az</code>	The true alt/az the dish was sent to
<code>drift_drive_alt</code> , <code>drift_drive_az</code>	The drive-grid point it parked on (Sect
<code>drift_crossing_time</code> , <code>drift_crossing_offset_deg</code>	When the source actually crosses the the second, and how far it misses centr
<code>homed_first</code> , <code>homing_count_error_alt_deg</code> , <code>homing_count_error_az_deg</code>	Whether the mount was homed just be what the counters read at the stops (Se

`read_h1_data.ipynb` reads these files, shows the metadata and plots waterfalls and mean spectra. All averaging is done in linear power; dB conversion is for display only.

10.4 Observation scheduler

`h1_web_scheduler.py` is a Flask application serving a tabbed interface on `http://localhost:5000`. Configuration persists in `scheduler_config.json` and activity is written to a rotating `scheduler.log`.

10.4.1 Interface

Scheduler tab

Add, edit, clone and delete observations, with clash prevention, late-start recovery, pre-emption, a countdown to the next slot, audio notification, JSON import/export and a “Clear Past” action. The list is always chronological with expired entries at the bottom; the running entry is green and queued ones amber, matched by identity (date, time and

name) rather than by name, because the simulator names entries by target and two can share one. Every row has an eye button that opens the entry in the simulator with the clock pinned to its moment; every button on the page has hover text. Running observations are protected from editing. The form's validation is its own (the modal is `novalidate`): the browser's constraint validation used to block a save silently when the offending field was one hidden by the chosen coordinate system.

Observe tab

Start an observation now — a tracked spectrum, a drift scan or a solar-flux track — and look at any recording: a dropdown of everything in `data/observations/` (date, filename, comment), the plot, a details table beside it (tuning, bands, whether the line was in band, calibration state, velocity frame, samples lost), *Fit model* (Section 9.4.3), *Download file*, and *View live recording*, which selects the file being written and re-plots it every 30 s. Drift scans and solar tracks get a live band-power trace while they run (Section 9.4.4); it comes down when the run ends and the recording's own plot takes its place.

RF calibration tab

The bandpass template and the counts-per-kelvin gain (Section 9.4.3), with suggested fields screened against the measured horizon, and a *Go to Lockman Hole* button for the empty field the bandpass wants.

Sun scan tab

Single Sun scan, calibration day, model fit, plot and apply — the workflow of Section 7.11.

Horizon tab

The horizon scan of Section 10.4.5, and the archive of past scans with the one in force chosen explicitly.

Simulator tab

The browser simulator of Section 11, embedded, with its *Schedule* button.

Configuration tab

Controller URL and fallbacks, observer location, slew and homing timeouts, Python and PlatformIO paths, receiver interpreter, data folder, minimum elevation, banner text and sound — and the fixed instrument (Section 10.1.1), editable here only, with a warning.

Log tab

Last N lines of `scheduler.log` with auto-refresh. The file captures DEBUG including periodic schedule dumps; the console is filtered to INFO. The log is the operational record of what the telescope did: the test suite detaches the loggers so that no test ever writes into it.

The page is plain static files — `web/index.html`, `web/app.css` and a dozen scripts under `web/js/` — served per request, so a change to it needs a browser refresh, not a scheduler restart. Until 25 August 2026 it was a 3184-line Python string inside the scheduler module, and a Python string applies Python's escapes to the JavaScript before the browser sees it, which silently killed every handler on the page twice in two days. `test_page_structure.py` guards the arrangement: every element the scripts look up exists, every inline handler names something callable, every `fetch` path is a real route, nothing in a blessed baseline has been lost, and every script parses (a syntax slip in one classic script kills its handlers with no message at all).

10.4.2 Runtime behaviour

- The scheduler re-executes itself under `receiver_python_path` (radioconda by default) unless it is already running there, so receiver subprocesses always have GNU Radio available.

- It is launched from the desktop **Start SRT Software** item via the VS Code workspace `folderOpen` task, which runs `start_scheduler.sh`. That wrapper reuses an already-serving instance rather than starting a second one that would die on the bound port. It is deliberately *not* a systemd service: a unit would contend for port 5000 with the task, and the scheduler should not come up unattended.
- A manual receiver boot is for warm-up and testing only. Scheduled observations stop it before claiming the SDR, and `/api/receiver/status` reports whether the running process is manual or observation-owned.
- A scheduled observation always wins: it cancels a running Sun scan or calibration day and waits up to `SUN_SCAN_PREEMPT_TIMEOUT` (600s) for the SDR before starting.
- `start_observation` claims the start briefly under `process_lock`, then does the pointing and slew-wait *unlocked* and abortable through `start_abort`. Blocking work must not be reintroduced under that lock.
- Failed or short-lived starts back off after `MAX_START_FAILURES` (3) attempts per schedule slot.
- CORS is restricted to the configured controller origins, so the scheduler buttons embedded in the controller's own page work only if `srt_controller_url` and its fallbacks match the origin that page is served from.
- `POST /api/config` accepts only keys already present in the default configuration, and observation filenames are constrained to stay inside the data folder.
- **One owner of the SDR and the mount.** Six things can claim the hardware — a scheduled observation, a Sun scan, a calibration day, a horizon scan, an RF calibration and the hand-started receiver window — and one function, `hardware_in_use()`, is the list of them. Each start path used to carry its own list and they had drifted; `test_exclusion.py` tests the matrix as a matrix, every pair both ways. A scheduled observation is deliberately outside the check, because it preempts rather than defers.
- **A finished slot is not started again.** Stopping a booked run, or running it to the end, retires its slot: the thread skips it while the slot is still nominally due. Without this a stopped observation restarted within ten seconds, twice in one day.
- A run started from the simulator with a live clock starts at once, through the same path as the Observe tab's Start Now, with its full duration, and is written to the schedule so the page shows it in its place. Booked, it waited for the next whole minute, then for the thread's 30s poll, and then had its duration trimmed to what was left of the slot.

10.4.3 Coordinate systems and drift scans

Table 23: Supported observation coordinate systems

System	Description	Tracking
Alt/Az	Direct horizontal pointing	Fixed position
RA/Dec (J2000)	Equatorial	Sidereal track
Galactic (l , b)	Galactic	Sidereal track
Solar-system object	Sun or Moon	Ephemeris track
Famous targets	A picker: H I fields, Cyg A, Cas A, Tau A, the Sun, the Moon	Track, or drift
Satellite (TLE)	Two-line element set	SGP4, 1 Hz
Drift	Fixed alt/az, source transits the beam	None, by design
Calibration day, horizon scan	Schedulable measurements	Own threads

A drift scan parks the dish at the alt/az a source will occupy at a chosen beam-crossing time T and lets the sky carry the source through the beam. `GET /api/drift_preview` returns the resulting pointing, reachability warnings against the horizon and the azimuth dead zone, and the source’s next meridian transit — the classical choice of T , because the source moves slowest in altitude there and the crossing is symmetric. The source may be a fixed RA/Dec or l/b , or a solar-system object (Sun, Moon, Jupiter) placed by the ephemeris at the crossing time; the Sun’s galactic coordinates were once computed by hand for a pointing test and came out pointing at the galactic centre, which is why the object path exists. The entry is stored as T and a half-window W ; $\text{start} = T - W$ and $\text{duration} = 2W$ are derived, and because the pointing is recomputed for each day’s T an entry that repeats daily stays centred. Since 26 August the dish parks on the *drive grid* point the source’s track passes closest to, and the moment it does is recorded (Section 7.8).

10.4.4 The measured horizon, in the schedule

Every entry carries *respect local horizon*, on by default. Almost everywhere the measured horizon of Section 10.4.5 is *advisory*: a profile can be months old, and stopping an observation on the word of a stale measurement is worse than knowingly taking a contaminated one, so the check reads the true alt/az from the controller after the slew and notes in the log if the beam is into the trees. Clearance asks for the full beamwidth above the measured floor, not the half — foliage at 1420 MHz is a ~ 290 K source, so a beam clipping it gains signal rather than losing it. Two exceptions act rather than warn. A **scheduled observation is trimmed** to the longest stretch with the target clear (the longest clear *run*, not merely clear at both ends: a circumpolar target can pass behind a tower between), idempotently, so rewriting the stored entry is safe; one with no clear stretch is marked and skipped. And a **Sun raster is refused** if any of its points is obstructed, because a contaminated point puts a ramp under the single Gaussian and pulls the centroid. Satellites, calibration days and horizon scans are exempt — the last necessarily, or it could never re-measure where the sky was last found blocked.

For satellites, PyEphem propagates the TLE with SGP4 [13] and the scheduler sends `/direct` updates every second. TLEs can be pasted, loaded from file, or fetched from CelesTrak by name or NORAD number.

10.4.5 The horizon scan

The measured horizon the checks above rely on is mapped radiometrically, because trees, the roofline and the two dome towers are ~ 290 K sources at 1420 MHz: lowering the beam through them is a step in total power of order the system temperature, not a marginal excess. Not

everything is — a metal-clad building has very low emissivity and reflects cold sky, showing only 2.3% of sky where the treeline gives 59% — so the scan judges an azimuth blocked by *significance* against radiometric noise, not by a fixed percentage, or it would call a building open sky, which is the unsafe direction.

The pattern is a **strip scan** (`horizon_strip_scan`): sweep azimuth at constant altitude, climbing in 5° steps from 5° . This moves the altitude axis once per strip, where an earlier per-azimuth cut scan moved it thousands of times and repeatedly into its lower limit — and lost the encoder counts of Section 7.9. Three defences come with it: the mount re-homes every second strip, bounding any count loss to one strip; a sky reference at a fixed high position is measured after each homing as a health check; and an azimuth found clear is dropped from every higher strip, because the horizon is monotonic, so the work peels away (70 azimuths to 24 to 7 to none) and the whole sky costs about 26 minutes. The clearing threshold is judged against three control azimuths that cleared on the first strip and are re-measured at every altitude, because clear sky low down is genuinely warmer through airmass and spillover, so neither the zenith nor the strip’s own distribution is a fair reference.

The horizon is seasonal — trees leaf out and are cut back — so scans are kept by date and never overwritten, under `horizon_profiles/`, with one named as in force. A finished scan is archived but *not* adopted automatically: choosing is a separate act, because a pruning genuinely opens the sky, so a fresh scan reads lower, and one taking effect on its own would drop the horizon with nobody having agreed the new measurement is the better record. The rule that decides whether a target is blocked samples the profile between its measured azimuths and takes the worse of the two, so the orange line drawn on the simulator is stepped, not smoothed — it claims no precision the 5° sampling does not have. With no profile measured, nothing is excluded: inventing a default would claim knowledge of a skyline nobody has surveyed. A horizon scan is itself a schedulable observation, so the twice-yearly measurement can be booked for a dark night.

10.4.6 Observation parameters

Table 24: Observation schedule parameters

Parameter	Description
Name	Descriptive name (a famous target supplies its own)
Comment	Free text, stored as the <code>comment</code> attribute of the recording
Start date/time	Local time; end time computed from the duration. Derived from <i>T</i> and <i>W</i> for a drift
Duration	Minutes
Coordinates	Target position in one of the systems above
Instrument	Shown, not set: the fixed instrument of Section 10.1.1
Integration time	Seconds per record
SDR type	B210, RTL-SDR or demo
Calibrator	Noise source on/off
Respect local horizon	Advisory check against the measured horizon; trims a scheduled entry
Home the mount first	Run the physical homing before pointing, and record the count error (Section 7.9)
When done	Stay, Home (run the physical homing sequence), or Stow (Alt 90, Az 180)

The tuning fields — centre frequency, bandwidth, gain, channels — were removed from the form with the fixed instrument; entries saved before it, or posted by a stale tab, are stripped of them on save. Note that “Home” here runs the Due’s homing sequence — a mechanical re-reference — while “Stow” parks at the drive position of Section 5. They are different operations and take different lengths of time.

10.4.7 Automated workflow

1. The user creates an observation in the web interface (or the simulator’s *Schedule* button does).
2. At the scheduled time — or at once, for a manual start — the scheduler first stops a hand-started receiver window, and cancels and waits for any Sun scan, calibration day, horizon scan or RF calibration holding the SDR.
3. If the entry asks for it, the mount is homed, with the counters read at the stops (Section 7.9); a homing that fails aborts the observation.
4. The pointing command goes to the controller. For a drift scan the parking point is chosen on the drive grid (Section 7.8).
5. It polls `/status` every 0.2s and waits for the reported drive position to match the drive target the controller holds, for two consecutive polls — not merely for `is_slewing` to clear, which is false for the first couple of seconds after any command. For a drift scan the true position the controller reports for the parked mount is then used to recompute the crossing time, model-free.
6. The measured horizon is consulted for the position actually reached; the calibrator noise source is set as configured.
7. For satellites, a background thread begins 1 Hz position updates.
8. The headless receiver is launched under the radioconda interpreter with the fixed instrument and the observation metadata in its environment, and its output goes to a log file beside the recording.
9. Both products are written to HDF5, in kelvin when the calibration applies, with a live sidecar for the plots.
10. On completion the calibrator is turned off, the slot is retired, and the “when done” action is carried out.

10.4.8 Tests

The scheduler and calibration code are covered by `pytest`:

```
1 cd receiver_scheduler && python -m pytest # ~490 tests, ~15 s, radioconda's Python
```

One known false failure: `TestFlaskAPI::test_post_config` fails on the observatory host because it makes a real HTTP call, reaches the live controller, and the controller then syncs the observer location back over the value the test just set. It is a test-isolation problem, unrelated to whatever is being changed; check it against a clean tree before chasing it.

Three rules the suite enforces on itself, each learned the hard way. **No test may open the radio:** one that stubbed the measurement but not the B210 session took the SDR from a running observation. **No test writes into the operational log or the last-observation pointer:** `conftest.py` detaches the file handlers and redirects the pointer for the session, after tests’ demo azimuths were once read as measurements. And **no test leaks files into the data folders:** horizon partials and recordings go to temporary directories.

11 Sky simulator

`astro_simulator/` predicts what this telescope should see. It is not part of the observing chain, but it is how an observation is planned and how a puzzling spectrum is checked against expectation — and it can hand its pointing to the real telescope.

11.1 Sky models

Two all-sky datasets back the simulator.

HI line

The HI4PI survey [6], with a $16.2'$ beam and 1.29 km s^{-1} channels — far finer than the 4.9° beam of Section 9.1, so the simulated spectrum is a genuine beam-weighted average of the survey brightness temperature rather than an interpolation.

1420 MHz continuum

The Stockert/Villa-Elisa survey [7], on the same grid, with the zero level (CMB plus isotropic, $\approx 3.2 \text{ K}$) stored separately and the strong sources — Cygnus A, Cassiopeia A, Taurus A — removed for analytic re-insertion, because they are saturated or blanked in the survey itself.

The full HI4PI cube is 33 GiB. The repository ships a compact form of about 23 MB, block-averaged to 0.5° pixels, smoothed to 1° total resolution, trimmed to $|v| \leq 470 \text{ km s}^{-1}$, thresholded at 3σ , quantised to 0.01 K in `int16` and LZMA-compressed. It is exact to better than 0.5% for beams of 1.6° and above, which covers every dish this simulator is meant for; the full cube is fetched from CDS only if a request falls outside what the compact data can honour.

11.2 Instrument model

The simulator computes the natural beam from the aperture exactly as in Section 9.1, convolves the survey to it, adds the continuum offset, and applies radiometer noise using Equation (48) — including the signal term, so channels on a bright line are correctly noisier than their neighbours.

Table 25: Simulator parameters and defaults

Option	Default	Unit	Meaning
<code>-dish</code>	3.0	m	Aperture; sets the beam as $1.22\lambda/D$
<code>-eta</code>	0.7	—	Main-beam efficiency
<code>-bw</code>	2.0	MHz	Bandwidth
<code>-nchan</code>	native	—	Spectrometer channels
<code>-tsys</code>	200	K	System temperature
<code>-tint</code>	60	s	Integration time
<code>-npol</code>	1	—	Polarisations summed
<code>-controller</code>	<code>http://192.168.50.120</code>	—	Target for <i>Realise</i> (desktop only)

The beam is not $1.22\lambda/D$ any more but the measured 5.164° of Section 9.1, shipped from `instrument.py` so the two simulators and the scheduler cannot disagree about it, and the drift-scan noise carries the measured total-power instability of Section 9.3.1 in quadrature with the radiometer floor — without it the simulated drift scan was thirteen times quieter than the receiver.

11.3 Velocity frames

The velocity axis can be expressed in the LSR, barycentric or topocentric frame, and `frame_offset()` is the reference implementation of the transformation discussed in Section 9.5. The receiver records topocentric frequencies and the simulator works natively in LSR; the plots and the gain fit both go through this one function, so the simulator is a practical check that the correction has been applied correctly.

11.4 Modes

The lower panel is modal, following the sky map selected above it.

Hi spectrum

With the HI map, clicking anywhere on the sky gives the dish spectrum for that direction, with editable pointing, beam, T_{sys} , integration time, channel count and velocity frame. A target list covers the standard HI pointings plus the continuum sources and the Sun and Moon; site horizon and visibility overlays and a live RA/Dec and alt/az readout show whether the direction is reachable from Glasgow at all.

Continuum drift scan

With the continuum map, the panel shows band-averaged T_A for the sky drifting through a beam parked on the current target — centred on beam-centre transit, over the duration set in the scan box, drawn as stepped measurements with per-sample noise. This is the planning tool for the drift observations of Section 10.

In the browser version the **band is not a parameter**: it follows the map type from the fixed instrument of Section 10.1.1 — the HI sub-band with the HI map, the continuum band (no hydrogen in it) with the continuum map — so the drift scan it draws is the continuum product a scheduled one will record, and the spectrum has the same 845 channels. A read-out beside the beam box states the band in force; what is offered depends on the mode, channels for a spectrum and scan length for a drift scan. The instrument block comes from `data/meta.json`, regenerated by `make_web_data.py -meta`, and a test checks the bundle against `tuning.py` so it cannot go stale silently.

11.5 The clock, the horizon and the map

The browser version has a pinnable clock: set a UTC time and the Sun, Moon, alt-0 loop, visibility loops and velocity frame are drawn for that moment; clear it and the sky runs live. The map greys out the sky below alt 0 and dims what lies behind the measured horizon of Section 10.4.5 when the scheduler serves the page, draws that horizon as the stepped orange line that *is* the rule deciding whether a target is blocked, marks the cardinal points on the alt-0 loop, and can be rotated by 90° in longitude to put the galactic centre at the edge. A *Reset* button returns every parameter to its default, keeping the pointing. Deep links carry the pointing in any frame (l/b, ra/dec, alt/az, target), the clock (time), the mode and the scan length, which is how the schedule’s eye button opens an entry as it will be observed.

11.6 Schedule (formerly Realise)

The browser version’s *Schedule* button hands the simulation to the scheduler rather than to the telescope directly: it posts the pointing, the mode, the scan length and the clock to `/api/simulator/schedule`, which builds an ordinary schedule entry — a tracked spectrum, or a drift scan starting at the clock and crossing beam centre half a scan in — with the comment “Observation set via the Simulator”. A pinned clock books it, at the next whole minute; a live clock *starts it now*, with its full duration, since “now” is not a booking. The button exists

only when the page is served by the scheduler, because that is the only origin from which the API can be reached; opened any other way it is not shown. The scheduler owns the astronomy of the drift case, so a page parameter never decides where the telescope points. The desktop simulator keeps its original *Realise* button, which commands the controller directly.

12 Installation

12.1 Prerequisites

- PlatformIO (VS Code extension or CLI)
- Python 3 with `esptool`
- Radioconda for the receiver and scheduler

PlatformIO is invoked by absolute path, since `pio` is not on `PATH`:

Platform	Binary
Windows	<code>~/.platformio/penv/Scripts/pio.exe</code>
Observatory Linux host	<code>~/.platformio/penv/bin/pio</code>

12.2 Arduino Due drive firmware

```

1 ~/.platformio/penv/Scripts/pio.exe run -e due           # build
2 ~/.platformio/penv/Scripts/pio.exe run -e due -t upload # flash
3 ~/.platformio/penv/Scripts/pio.exe run -e simulation    # host-side simulation
4 ~/.platformio/penv/Scripts/pio.exe device monitor -b 115200

```

12.3 WT32-ETH01 controller firmware

```

1 cd esp32_controller_arduino
2 ~/.platformio/penv/Scripts/pio.exe run                # wt32-eth01 (
  default)
3 ~/.platformio/penv/Scripts/pio.exe run -e wt32-eth01 -t upload # first, serial
  flash
4 ~/.platformio/penv/Scripts/pio.exe run -e wt32-eth01-ota -t upload # routine,
  over Ethernet
5 ~/.platformio/penv/Scripts/pio.exe test -e native      # host unit tests

```

The WT32-ETH01 is the deployed board and the default build; it must always compile clean. The `esp32s3` environment is legacy — that board is no longer used — and is kept building only when convenient.

12.4 Configuration

The observer location defaults live in the controller's `config.h` and are the *true* site position:

```

1 #define OBSERVER_LAT 55.902426 // degrees north
2 #define OBSERVER_LON -4.307865 // degrees east (west negative)

```

They can be changed at runtime from the Settings tab and are persisted to NVS. The same numbers appear in the scheduler's default configuration and in `sun_scan.py`, and all three must agree. Nothing should ever write a *fictitious* observer position to compensate for pointing error; that is what the pointing model is for.

12.5 Receiver prerequisites

```

1 # 1. Install radioconda: https://github.com/ryanvolz/radioconda/releases
2 # 2. Activate it
3 conda activate radioconda
4 # 3. Install the remaining Python dependencies
5 cd receiver_scheduler
6 pip install -r requirements.txt

```

Radioconda supplies GNU Radio, PyQt5, NumPy, SciPy and h5py; `requirements.txt` adds Flask, PyEphem and pytest.

13 Testing and verification

There are four ways to exercise this system without a telescope, and between them they cover almost everything except the motors themselves. Anyone changing the code should know which one applies before starting.

Table 26: Test paths

Path	Covers	Command
Drive simulation	Due firmware	<code>pio run -e simulation</code>
Native unit tests	Coordinate and pointing maths	<code>pio test -e native</code>
Pytest	Scheduler, Sun scan, model fit	<code>pytest receiver_scheduler/</code>
Due emulator	Controller firmware on the bench	<code>python tools/due_emulator.py -port COM7</code>

13.1 Drive firmware in simulation

Described in Section 4: GPIO, PWM, ADC and interrupts are replaced by macros and `simulatePulses()` advances position from the commanded PWM. Both the `due` and `simulation` environments must build clean, and any change to the homing flow or to in-loop motor control must still complete in simulation — the homing while-loops call `simulatePulses()` themselves precisely so that they can.

13.2 Native unit tests

The `native` environment compiles `coordinates.cpp` and `pointing.cpp` on the host and runs `test/test_coordinates` and `test/test_pointing` against them — around 110 assertions covering Julian dates, sidereal time, precession, the horizontal and galactic transforms, the ephemerides, the pointing model terms, and the round-trip identity `driveToTrue(trueToDrive(x)) = x`.

The important property is that these are the *flashed* sources, not copies: `test/shim/` supplies only the small parts of `Arduino.h` and `Preferences.h` that `pointing.cpp` touches. A test that passes against a transcribed copy of an algorithm proves nothing about the firmware.

13.3 Scheduler and calibration tests

```

1 python -m pytest receiver_scheduler/test_scheduler.py receiver_scheduler/
   test_sun_scan.py

```

Roughly 156 tests covering scheduling logic, clash and late-start handling, the sun-scan geometry, the design matrix — including a check of the pointing-model matrix against numerically rotated horizon coordinates — the fit, the quality gates and the wire format. The known false failure of `TestFlaskAPI::test_post_config` on the observatory host is described in Section 10.

13.4 The Due emulator

`tools/due_emulator.py` presents the Due end of the UART protocol over a 3.3 V USB-to-UART adapter, so the controller firmware can be exercised on the bench with no telescope attached. It reproduces the 1 Hz status stream in the exact positional format, the full command set, quantised motion at 0.5° , homing, and plausible motor currents.

The ESP32 is not 5 V tolerant. Set the adapter jumper to 3.3 V, wire TX and RX crossed to IO4 and IO14 with a common ground, and do not use the board's TX0/RX0 pins — those are the programming console.

Its value is in the failures it can inject, which are exactly the ones that are hard to produce deliberately on a working telescope:

Table 27: Emulator fault-injection commands

Command	Effect
<code>fault <text></code>	Enter FAULT with the given message
<code>clear</code>	Clear the fault, as a power cycle would
<code>pos <alt> <az></code>	Teleport the reported position
<code>quiet on off</code>	Stop or resume the status stream, testing the timeout UI
<code>garbage</code>	Send one malformed line, testing the parser
<code>split</code>	Send the next status line in two delayed chunks

`split` deserves particular note: line splicing on the UART is what produced the corrupt status readings that led to the rate limit of Section 4, and it is otherwise reproducible only by overloading a real link. The `-capture` option logs every raw chunk with timestamps, which is how interleaved command lines from the controller are detected.

The issues labelled **needs-hardware** cannot be closed on a development host and need either this emulator, a bench ESP32, or the mount itself.

14 Operation

14.1 Startup sequence

1. Power on the Arduino Due and the controller.
2. The Due zeroes its current sensors, then runs the three-phase homing sequence of Section 4.3, finishing at the configured home position and reporting **Ready**. Allow 30–60 s.
3. The controller brings up Ethernet, obtains its pinned DHCP address, starts the WiFi AP, loads the settings and the stored pointing model from NVS, and begins SNTP.
4. Confirm on the web interface that the clock has synced and that the pointing model is loaded (`pointing_loaded` in `/status`).

A controller that comes up with no pointing model is not broken — it applies the identity transform plus refraction — but it will point about two degrees off, which is most of a beam width.

14.2 Reaching the web interface

Normally, from the observatory computer, either `http://192.168.50.120/` or the mDNS name `http://srt-controller.local/`.

If that fails, connect to the WiFi network `SRT_Controller` with password `radio1420`, and browse to `http://192.168.4.1`. The AP is up regardless of the Ethernet configuration and is the standing way back in.

14.3 Observing

1. Enter the target: RA (0–24 h) and Dec (–90 to +90°); galactic l (0–360°) and b ; or alt/az.
2. “Track” for continuous tracking, “Go To” for a single slew.
3. One-click targets are provided for the Sun, the Moon, the galactic plane (Section 5.6.3) and stow.
4. Watch the status readout: it shows the drive position, the true sky position it corresponds to, the derived RA/Dec and l , b , and whether the mount is slewing.

Targets below the observing horizon are refused outright; a tracked target that sets parks the dish at stow and resumes automatically on rising.

14.4 Safety features

- **Two-tier position limits** — software limits inside the physical stops, with a bounded transient tolerance
- **Current limiting** on the filtered current, with continuously re-tracked zero offsets
- **Stall detection** requiring both a settled drive start and an absence of pulses
- **Fault-flag persistence** filtering, so switching transients do not trip a fault
- **Pointing-correction clamps**, so a broken model cannot fling the mount at a limit switch
- **FAULT latching**, requiring an explicit reset

15 Troubleshooting

15.1 System does not start

Symptom	Check
No serial output	USB cable, correct port
Stuck in homing	Motor and encoder connections
Immediate fault	Driver power, fault-flag wiring
Homing aborts “not responding”	Encoder wiring, or <code>DEBOUNCE</code> set too long

15.2 Motors do not move

Symptom	Check
PWM output but no motion	Driver reset pins (should be HIGH)
No PWM output	PWM pins 8 and 10
Motors run backwards	Direction wiring, and the <code>*_DIR_INVERT</code> macros

15.3 Position errors

Symptom	Check
Wrong position after homing	HOMEALT/HOMEAZ
Position drifts	Encoder debounce; increase DEBOUNCE
Total pulse loss at speed	Debounce window comparable to the pulse interval
Overshoots target	Increase RAMPDOWN
Jerky motion	Increase RAMPUP

15.4 Pointing errors

Symptom	Check
Consistent $\sim 2^\circ$ offset	<code>pointing_loaded</code> false — no model applied
Offset returned after a reboot	Model applied to the offset boxes, not <code>/pointing/apply</code>
Error grows through the day	Clock; check <code>sync_count</code> and <code>last_sync_age_s</code>
Fit refused, low significance	Scans span too little azimuth; add a morning <i>and</i> afternoon set
Fit refused, high χ^2_ν	Something other than tilt is moving the pointing — check for mechanical slip
Altitude error doubles at low elevation	Refraction counted twice: the two <code>refraction</code> implementations have diverged
Dish parks at drive azimuth 170	Stow being routed through the pointing model

15.5 Network and web interface

Symptom	Check
Cannot reach 192.168.50.120	<code>srt-link</code> up, cable on the second card, DHCP lease in the journal
Reachable but the page has no data	Due serial link and baud rate
Scheduler buttons dead on the controller page	CORS: <code>srt_controller_url</code> must match the page's origin
OTA “No response from device”	Missing <code>ufw allow in on enp5s0</code> for the ephemeral callback
Clock never syncs	Missing <code>ufw route allow</code> rule; NAT is being dropped
<code>dns_after_wifi</code> is 0.0.0.0	Expected; lwIP clears the resolver list when the AP starts
<code>malformed_status</code> climbing	Serial1 status flooding; check the rate limit in <code>outputStatus()</code>

15.6 Coordinates and Stellarium

Symptom	Check
Position does not match the sky	Observer latitude and longitude, and time sync
Errors above 1°	Time sync status first, pointing model second
Sun/Moon out by ~ 20 arcmin	Normal for the simplified lunar ephemeris
Stellarium will not connect	Address and port 10001
Connects but will not slew	Select the object, then Ctrl+1
Stellarium marker offset from the dish	<code>driveToTrue()</code> not being applied to the reported position

16 Outstanding work and known limitations

This document describes a working instrument, not a finished one. Two categories of open item are worth stating plainly so that the preceding sections are not read as a claim of completeness.

16.1 Known limitations

The recorded velocity axis is topocentric

Section 9.5. The plots draw an LSR axis and say so; anyone reducing the files directly must apply the correction themselves, and it is not small.

The temperature scale rests on the H i model

Section 9.4.3. The gain and T_{sys} are fitted against the simulator's prediction of the line, which is circular for line science and separates the two only through the line's contrast. The noise diode is switched and recorded but not yet used; the way forward — a receiver-generated pseudorandom comb radiated from a vertex antenna and recovered by cross-correlation, giving gain, passband and T_{sys} per record — is issue #26.

The beam width is the largest systematic in that scale

Section 9.1.1. Three independent measurements on 26 August 2026 put the beam near $4.0\text{--}4.6^\circ$ rather than the 5.164° the model assumes, and the assumed value moves the fitted gain by 6% and T_{sys} by 22 K. Unresolved; the test is continuous drift scans of Cyg A, Tau A and the Sun at several elevations.

The Cas A excess

A drift scan of Cassiopeia A read 1.46 times the model with a 29-minute crossing against 40 predicted — consistent with a narrower beam, and possibly with a point-source response that is higher at the zenith than the Sun scans see at $20\text{--}50^\circ$. Unresolved, and bound up with the item above.

Pointing model fitted from the Sun alone

Six terms are now constrained (IE, IA, AN, AE, CA and an azimuth scale) from 49 scans over three days; NPAE and TF exist in the transform and the wire format but are left at zero. A source list spanning a wider range of elevations would constrain the rest.

Moon tracking is geocentric

No parallax, up to 0.9° low — to be fixed before the Moon is used as a calibrator. The simplified lunar ephemeris adds 5–10 arcminutes.

Encoder counts are only as good as the last homing

Section 7.9. Counts were lost twice in a week; the homing-first option measures the loss, and the Due printing its counter at the stall (issue #24) is the belt-and-braces version.

No inbound route to the controller from the site LAN

Section 8, deliberately.

16.2 Where the open work is recorded

Outstanding work lives in GitHub issues on the repository, not in this document and not in TODO files — two earlier review documents were folded into issues and removed, and their content remains in the git history. Each issue carries the finding, `file:line` references, a suggested approach and its verification steps.

Issues labelled **needs-hardware** cannot be verified on a development host: they need a bench ESP32, the Due emulator of Section 13, or the mount itself, and elsewhere are compile-check-only. The most significant open item at the time of writing is the missing cross-task locking in parts of the controller firmware.

Several issues record conclusions that took substantial analysis and are easy to re-derive incorrectly — in particular, that no tracking lead should be added because rounding already performs it, and that scans must record absolute (T, D) pairs rather than composing deltas (Section 7). The reasoning is stated in each issue; re-deriving from first principles tends to reach the plausible-but-wrong version.

17 The 21 cm line: expected signals and the temperature scale

Section 9 established the beam and the units. This section is about the astronomy those units are for: what the 21 cm line is, what antenna temperature this particular telescope should see from it, what system temperature to expect, and how to measure that system temperature using nothing but the sky and the Sun.

17.1 The hyperfine transition

Neutral atomic hydrogen in its ground state has two hyperfine levels, according to whether the proton and electron spins are parallel ($F = 1$) or antiparallel ($F = 0$). The splitting corresponds to

$$\nu_0 = 1420.405751768 \text{ MHz}, \quad \lambda_0 = 21.106 \text{ cm}. \quad (56)$$

The transition is magnetic-dipole and strongly forbidden [8]: the Einstein coefficient is

$$A_{10} = 2.869 \times 10^{-15} \text{ s}^{-1}, \quad (57)$$

so a single atom radiates about once every 11 million years. It is observable at all only because the Galaxy contains of order $10^{10} M_\odot$ of neutral hydrogen, and because collisions keep the two levels close to thermal equilibrium at a *spin temperature* T_s , which in the cold neutral medium tracks the kinetic temperature.

The level populations follow $n_1/n_0 = 3 \exp(-h\nu_0/kT_s)$, and since $h\nu_0/k = 0.068 \text{ K}$ is far below any astrophysical temperature, the ratio is 3 to within a part in a thousand: three quarters of all H I atoms are in the upper state at any time, regardless of temperature. This is why 21 cm emission measures *how much* hydrogen there is, almost independently of how hot it is.

17.2 What the line measures

The observed brightness temperature along a line of sight is

$$T_B = T_s (1 - e^{-\tau_\nu}), \quad (58)$$

and for the optically thin case ($\tau_\nu \ll 1$), which holds for most directions away from the inner Galactic plane, this reduces to $T_B \approx T_s \tau_\nu$ and the integral over the line profile gives the column density directly:

$$\left[\frac{N_{\text{H I}}}{\text{cm}^{-2}} \right] = 1.823 \times 10^{18} \int \left[\frac{T_B}{\text{K}} \right] \left[\frac{dv}{\text{km s}^{-1}} \right]. \quad (59)$$

Equation (59) [9] is the single most useful result in 21 cm astronomy, and it is the reason spectra must be stored in linear power and calibrated in kelvin rather than decibels: the integral of a logarithm means nothing.

Table 28: Typical H I line parameters for the directions this telescope can reach

Direction	Peak T_{B} (K)	$\int T_{\text{B}} dv$ (K km s ⁻¹)	$N_{\text{H I}}$ (cm ⁻²)
Inner Galactic plane, $b = 0$	80–120	500–1600	$1\text{--}3 \times 10^{21}$
Outer plane / anticentre	50–80	300–700	$0.5\text{--}1.3 \times 10^{21}$
Intermediate latitude, $ b \sim 30^\circ$	5–15	100–300	$2\text{--}5 \times 10^{20}$
Coldest high-latitude sky	1–3	30–80	$0.6\text{--}1.5 \times 10^{20}$

Two consequences of the 4.9° beam are worth stating plainly. First, this instrument never resolves individual clouds: it measures the beam-averaged column density over a patch of Galaxy some 5° across, which at the distance of the inner Galaxy is hundreds of parsecs. Second, the line profile it records is the superposition of everything along the line of sight, separated only in velocity — which is precisely what makes the next subsection possible.

17.3 Galactic rotation and the tangent-point method

Because the line is intrinsically narrow, its observed frequency encodes the radial velocity of the emitting gas, and Galactic rotation converts that velocity into position. For gas in circular rotation in the plane at galactocentric radius R , observed at Galactic longitude l ,

$$v_{\text{LSR}} = R_0 \sin l \left[\frac{V(R)}{R} - \frac{V_0}{R_0} \right], \quad (60)$$

with $R_0 \approx 8.2$ kpc the Sun’s galactocentric distance and $V_0 \approx 235$ km s⁻¹ the local circular speed.

In the first quadrant ($0^\circ < l < 90^\circ$) the line of sight reaches a minimum galactocentric radius, the *tangent point*, at $R = R_0 \sin l$, where the whole circular velocity lies along the line of sight. That gas produces the extreme positive velocity in the profile, and Equation (60) collapses to

$$V(R_0 \sin l) = v_{\text{max}}(l) + V_0 \sin l. \quad (61)$$

Measuring the terminal velocity v_{max} as a function of longitude therefore traces the Galactic rotation curve directly, with no distance ladder and no assumption beyond circular rotation. It is the classic experiment for a telescope of this size, and everything in this document exists to make it possible: the pointing must be good to a fraction of the beam so that l means what it says, the clock must be right so that the alt-az conversion does not smear the pointing (Section 5), and the velocity axis must be corrected to the LSR (Section 9.5) or the derived rotation curve acquires an annual wobble of up to 30 km s⁻¹.

A bandwidth of 2 MHz spans ± 211 km s⁻¹ and the default 2.4 MHz spans ± 253 km s⁻¹, comfortably covering the ± 150 km s⁻¹ that Galactic rotation produces. The default channel width of 586 Hz is 0.124 km s⁻¹, some eighty times finer than the ~ 10 km s⁻¹ width of a typical line component — deliberately, because channels can be averaged after the fact but never un-averaged, and smoothing to 10 km s⁻¹ buys a factor of nine in sensitivity.

17.4 What the telescope collects

The geometric area of a 3.0 m dish is $A_g = \pi D^2/4 = 7.07$ m². The effective area follows from the beam through the antenna theorem, $A_e \Omega_A = \lambda^2$, with $\Omega_A = \Omega_{\text{MB}}/\eta_{\text{MB}}$:

$$A_e = \frac{\lambda^2 \eta_{\text{MB}}}{1.133 \theta_{\text{FWHM}}^2} = \frac{0.04455 \times 0.7}{1.133 \times (0.0858)^2} = 3.74 \text{ m}^2, \quad (62)$$

an aperture efficiency of $\eta_A = A_e/A_g = 0.53$ — a reasonable value for a prime-focus dish with a simple feed, and the number the Sun measurement of Section 17.8 can check. The sensitivity in kelvin per jansky is then

$$\Gamma = \frac{A_e}{2k} = \frac{3.74}{2 \times 1.381 \times 10^{-23}} = 1.35 \times 10^{-3} \text{ K Jy}^{-1}, \quad (63)$$

that is, **1.35 mK per jansky**. The reciprocal quantity, the system equivalent flux density, is

$$\text{SEFD} = \frac{2k T_{\text{sys}}}{A_e} = \frac{T_{\text{sys}}}{\Gamma} \approx 5.2 \times 10^4 \text{ Jy} \quad \text{for } T_{\text{sys}} = 70 \text{ K}. \quad (64)$$

Fifty thousand janskys is a large number, and it correctly conveys that this is a small telescope: only the handful of brightest radio sources in the sky are detectable at all in continuum.

17.5 Expected antenna temperatures

For a source small compared with the beam, $T_A = f \Gamma S$, where f is the beam-coupling factor. For a uniform disc of angular diameter θ_s observed with a Gaussian beam of width θ_b ,

$$f = \frac{1 - \exp(-\ln 2 \cdot x^2)}{\ln 2 \cdot x^2}, \quad x = \frac{\theta_s}{\theta_b}, \quad (65)$$

which for the Sun or Moon ($\theta_s = 0.53^\circ$, $x = 0.108$) gives $f = 0.996$ — a 0.4% correction, negligible beside the other uncertainties but free to apply. For a source that fills the beam, such as the H I line itself, the relation is instead $T_A = \eta_{\text{MB}} T_B$.

Table 29: Expected antenna temperatures, using $\Gamma = 1.35 \text{ mK Jy}^{-1}$ and $\eta_{\text{MB}} = 0.7$. The Y column is the total-power ratio against blank sky for $T_{\text{sys}} = 70 \text{ K}$.

Source	Type	S or T_B	T_A (K)	Y
Sun (quiet)	disc, 0.53°	$5.0 \times 10^5 \text{ Jy}$	677	10.7
Sun (active)	disc	$10\text{--}100 \times \text{quiet}$	$10^4\text{--}10^5$	—
Cassiopeia A	compact	1660 Jy (2026)	2.25	1.032
Cygnus A	compact	1590 Jy	2.15	1.031
Moon	disc, 0.52°	890 Jy	1.20	1.017
Taurus A	compact	875 Jy	1.18	1.017
H I, inner plane	fills beam	80–120 K	55–85	1.8–2.2
H I, high latitude	fills beam	1–3 K	0.7–2.1	1.01–1.03
Galactic continuum, plane	fills beam	10–30 K	7–21	1.1–1.3

Cassiopeia A fades secularly at about 0.67% per year in L band, so its flux is computed from the Baars [10] epoch-1965 value with the Trotter [11] rate rather than tabulated; the simulator does exactly this, and the 2026 figure above comes from it.

Note the enormous dynamic range in that table. The Sun raises the total power by an order of magnitude; the next brightest object in the sky raises it by three per cent. That asymmetry matters more than it first appears, and the next subsection explains why it disqualifies almost everything in the table as a calibrator.

17.6 Why only the Sun and Moon can calibrate a 5° beam

A source list containing Cassiopeia A, Cygnus A and Taurus A invites the assumption that a small dish has four or five usable calibrators. It does not. At 4.9° the beam collects some 27 square degrees of sky, and all three of those sources lie within a few degrees of the Galactic plane,

where the diffuse 1420 MHz continuum is far from negligible. **Pointing at them measures the Galactic background as much as the source.**

Table 30 makes this concrete, using the Stockert/Villa-Elisa survey that this repository already ships — with the strong sources removed from it, so the background column is genuinely diffuse emission — convolved to the telescope’s own beam.

Table 30: Diffuse Galactic continuum in a 4.9° beam at the calibrator positions, from the compact 1420 MHz survey. The last column is the spread in background over eight off-positions one beamwidth away — the systematic an on/off measurement inherits from its choice of reference.

Source	l, b	T_A source	T_A background	Off-position spread
Cassiopeia A	$111.7^\circ, -2.1^\circ$	2.25 K	1.33 K	0.79 K
Cygnus A	$76.2^\circ, +5.8^\circ$	2.15 K	1.77 K	4.03 K
Taurus A	$184.6^\circ, -5.8^\circ$	1.18 K	0.70 K	0.16 K
Sun	(ecliptic)	677 K	< 5 K	< 5 K
Moon	(ecliptic)	1.20 K	< 5 K	<i>exactly zero</i>

The background is between 0.6 and 0.8 times the source signal in every case, and the off-position spread — the part that does not cancel — runs from 14% of the source for Taurus A to 190% of it for Cygnus A, which sits beside the Cygnus X star-forming complex. Cygnus A is simply unusable: whichever reference position is chosen, the answer changes by more than the thing being measured. Cassiopeia A carries a 35% systematic from the same cause, and Taurus A, at the anticentre, is the least bad of the three at 14% — still an order of magnitude worse than the 6–12% the solar method achieves.

The distinguishing property is not brightness, it is whether the background can be removed. A source fixed on the celestial sphere can never supply a reference measurement of the same piece of sky, because there is no epoch at which it is absent; every off-position is a different patch of Galaxy, and at this resolution those patches differ by a kelvin or more. The Sun and Moon escape by two different routes:

The Sun escapes by brightness

At 677 K against a background of at most a few kelvin anywhere on the ecliptic, the contamination is below a per cent. No background subtraction is needed at all.

The Moon escapes by motion

At 1.20 K the Moon is no brighter than Taurus A, and on its own that would make it equally useless. But it moves about 13° per day — a beamwidth every nine hours — so the *same celestial coordinates* can be observed with the Moon in the beam and again once it has left. The background then cancels identically rather than approximately, which is precisely what the fixed sources cannot offer. It is also a black body of known angular size at a physical temperature near 225 K, so its flux is predictable rather than measured.

Extragalactic source confusion, incidentally, is not the problem here: for a beam this size the classical confusion noise is of order 10^{-2} K, two orders below the diffuse Galactic variation. It is the Milky Way’s own emission that spoils these calibrators, not the population of faint sources behind them.

The practical consequence is stated once and applies throughout the rest of this section: *the Sun is the calibrator, the Moon is the independent check, and the fixed radio sources are useful only as tests of gain stability* — where a repeatable on/off difference matters and its absolute value does not.

17.7 Expected system temperature

Adding the budget of Section 9 to the sky contributions above:

Table 31: Expected system temperature in different circumstances

Condition	T_{sys} (K)	Dominated by
Cold sky, high latitude, near zenith	66–70	Receiver (43 K) and spillover (17 K)
Same, at 20° elevation	80–90	Ground spillover, atmosphere (sec z)
Galactic plane, continuum only	75–90	Galactic synchrotron in the beam
Galactic plane, at H I line centre	130–160	The line itself
Beam on the quiet Sun	~ 750	The Sun
Beam on an active Sun	10^4 – 10^5	The Sun, and possibly saturation

The atmosphere contributes $T_{\text{atm}}(1 - e^{-\tau_0 \sec z})$, and at 1.4 GHz the zenith opacity is only $\tau_0 \approx 0.01$, so with $T_{\text{atm}} \approx 275$ K this is about 2.7 K at the zenith rising to 8 K at 20° elevation. It is a small term, which is one of the pleasant features of observing at this frequency, but it is not zero and it is elevation-dependent — so a system temperature quoted without an elevation is incomplete.

With $T_{\text{sys}} = 70$ K the radiometer equation gives, per 586 Hz channel in 60 s, $\Delta T = 0.37$ K; smoothed to 10 km s^{-1} resolution the same minute gives 0.04 K. A high-latitude H I profile of 1–3 K peak is therefore a solid detection within a minute, and a well-measured profile within an hour.

17.8 Measuring the system temperature

A total-power measurement gives

$$P = G (T_{\text{sys}} + T_{\text{A}}), \quad (66)$$

with two unknowns, the gain G and T_{sys} , and one equation. *One pointing can never determine the temperature scale.* Every method below is a way of obtaining a second, independent equation, and the differences between them are entirely about what has to be assumed.

17.8.1 Method A: ambient absorber

The most direct measurement of the receiver alone. Place microwave absorber at ambient temperature over the feed, then move it aside and look at cold sky:

$$Y = \frac{P_{\text{hot}}}{P_{\text{cold}}} = \frac{T_{\text{amb}} + T_{\text{rx}}}{T_{\text{cold}} + T_{\text{rx}}}, \quad T_{\text{rx}} = \frac{T_{\text{amb}} - Y T_{\text{cold}}}{Y - 1}. \quad (67)$$

With $T_{\text{amb}} = 290$ K and $T_{\text{cold}} \approx 6$ K (sky plus atmosphere), a receiver at 43 K gives $Y = 6.79$. The measurement is accurate and needs no astronomy at all, but note what it does *not* include: the absorber covers the feed, so spillover and ground pickup — the 17 K term, and the one most likely to be wrong — are excluded. It measures T_{rx} , not T_{sys} .

17.8.2 Method B: the Sun, of known flux density

This is the method this telescope is built for, since it already knows how to find and scan the Sun (Section 7). Compare the total power on the Sun with that on nearby blank sky:

$$Y_{\odot} = \frac{P_{\odot}}{P_{\text{sky}}} = 1 + \frac{T_{\text{A},\odot}}{T_{\text{sys}}}, \quad T_{\text{sys}} = \frac{f \Gamma S_{\odot}}{Y_{\odot} - 1}. \quad (68)$$

There is a subtlety here worth extracting, because it decides how much the result can be trusted. Substituting Equation (63) into (68) and comparing with Equation (64) gives

$$\boxed{\text{SEFD} = \frac{f S_{\odot}}{Y_{\odot} - 1}} \quad (69)$$

— the effective area cancels entirely. **The Sun measures the SEFD directly, with no assumption about aperture efficiency**, and the SEFD is the quantity that actually sets what the telescope can detect. Converting that to a system temperature in kelvin, by contrast, requires A_e , and it is η_A that carries the largest systematic error in the chain. The honest way to use this measurement is therefore to quote the SEFD as the primary result, and to treat T_{sys} and η_A as a pair — fix either one and the Sun gives you the other.

Four corrections and cautions apply.

The solar flux must be contemporaneous, not assumed

The quiet Sun radiates about 5×10^5 Jy (50 solar flux units) at 21 cm, but an active Sun is ten to a hundred times brighter and varies on timescales of minutes. Fortunately the standard monitoring frequencies include 1415 MHz — the RSTN stations and DRAO both publish it daily [12] — which is within 0.4% of the observing frequency, so no spectral extrapolation is needed. Using a nominal quiet-Sun value on a day with an active region is the largest avoidable error in this method.

Point accurately, and do it after the pointing calibration

A pointing offset Δ reduces the response by $\exp(-4 \ln 2 \Delta^2 / \theta_b^2)$, which is 2.8% at $\Delta = 0.5^\circ$ and 11% at 1° . An uncalibrated mount, mispointing by the 2° of Section 7, would under-read by 40%. Better still, use the peak of a Sun raster rather than a single pointing — the machinery already exists.

Take the reference close by in both angle and time

The blank-sky reference should be a few degrees off the Sun and taken within a minute or two, so that gain drift, elevation-dependent spillover and atmospheric path all cancel rather than being corrected.

Correct for atmospheric attenuation

The Sun is attenuated by $e^{-\tau_0 \sec z}$ on the way in, about 1% at the zenith and 3% at 20° . Small, but it acts in the same direction as the pointing error.

17.8.3 The two-point calibration: blank sky and the Sun as loads

The Y-factor of Equation (68) extracts one number, T_{sys} , from the ratio of two measurements. The same two pointings, treated instead as two *known loads*, do considerably more: they establish the whole power-to-temperature scale, which is what turns the receiver's arbitrary linear units into kelvin.

Write the detected power as a linear response to the input temperature,

$$P = g T_{\text{in}} + P_0, \quad (70)$$

with g the gain in units per kelvin and P_0 any offset that is present when no sky power is. Blank sky and the Sun give two points on that line:

$$P_{\text{sky}} = g T_{\text{sys}} + P_0, \quad (71)$$

$$P_{\odot} = g (T_{\text{sys}} + T_{A,\odot}) + P_0, \quad (72)$$

and subtracting removes the offset entirely, leaving the scale factor as a pure difference measurement:

$$k \equiv \frac{1}{g} = \frac{T_{A,\odot}}{P_{\odot} - P_{\text{sky}}} = \frac{f \Gamma S_{\odot}}{P_{\odot} - P_{\text{sky}}} \quad [\text{K per unit}] \quad (73)$$

The blank-sky level then reads

$$T_{\text{sys}} = k (P_{\text{sky}} - P_0), \quad (74)$$

and *this* is where the offset matters. Equation (73) is immune to P_0 ; Equation (74) is not, and neither is the Y -factor, which tacitly sets $P_0 = 0$. If the detector carries a pedestal — a spectrometer noise floor, a DC spur at band centre, a quantisation offset — then the Y -factor overestimates the system temperature by exactly kP_0 . The two measurements alone cannot reveal this; a third point can, and that is the argument for combining methods rather than choosing one.

Worked example. Suppose blank sky reads $P_{\text{sky}} = 1.000$ in the receiver’s arbitrary units and the Sun raster peak reads $P_{\odot} = 10.70$, on a day when the 1415 MHz solar flux is 50 sfu. Then $T_{A,\odot} = f \Gamma S_{\odot} = 0.996 \times 1.35 \times 10^{-3} \times 5.0 \times 10^5 = 673 \text{ K}$, and

$$k = \frac{673}{10.70 - 1.000} = 69.4 \text{ K per unit}, \quad T_{\text{sys}} = 69.4 \times 1.000 = 69 \text{ K}, \quad (75)$$

assuming $P_0 = 0$. That is the measurement the receiver’s calibration box currently *assumes*: its single-point calibration (Section 9) asks the operator to type in a blank-sky temperature and derives $k = T_{\text{cal}}/\bar{P}$ from it. The two-point procedure turns that entry from an input into an output — the same arithmetic, but with the 70 K established by observation rather than by a comment in `b210_h1_receiver.py`.

Applying the scale to spectra. For spectroscopy the two-point result is used in the standard position-switched form. With $P_{\text{on}}(\nu)$ on source and $P_{\text{off}}(\nu)$ on a nearby emission-free reference,

$$T_A(\nu) = T_{\text{sys}} \frac{P_{\text{on}}(\nu) - P_{\text{off}}(\nu)}{P_{\text{off}}(\nu)}. \quad (76)$$

This form is worth more than it looks. Dividing by $P_{\text{off}}(\nu)$ removes the instrumental bandpass — which is emphatically not flat across 2.4 MHz — at the same time as it sets the scale, and being a ratio it is again free of P_0 and of any slow change in g between the two. The two-point sky-and-Sun measurement supplies the single number T_{sys} that Equation (76) needs, and nothing else in the chain requires absolute power at all. This is the practical calibration pipeline for this telescope: measure T_{sys} against the Sun once per session, then observe H I entirely in on/off ratios.

Check the linearity before trusting either. Blank sky and the Sun differ by a factor of about ten in input power, whereas the H I line the calibration will be applied to sits within a factor of two of blank sky. A scale derived at the top of that range and applied at the bottom is only valid if the detector is linear across it. Two precautions follow, and both are cheap:

- **Fix the SDR gain and disable any automatic gain control.** The receiver already sets gain explicitly rather than leaving AGC enabled, but a gain chosen for H I may put the ADC near full scale on the Sun, and a compressed solar reading inflates k and deflates T_{sys} . Confirm the Sun measurement is not clipping before using it.
- **Take a third point in between.** It must be one whose input temperature is genuinely known, which rules out the fixed radio sources for the reason given in Section 17.6. Two qualify: the Moon ($\sim 1.2 \text{ K}$, differenced against the same sky once it has moved on) and

the Galactic plane continuum ($\sim 5\text{--}20\text{ K}$, read from the survey and convolved to the beam as in Method C). Three points falling on a straight line through the origin validate both the linearity and the assumption $P_0 = 0$; three on a straight line with a non-zero intercept give the offset, after which Equation (74) can be applied properly.

Adding the ambient absorber of Method A as the hot point and blank sky as the cold one gives the same two-point structure with a load whose temperature is known to a fraction of a kelvin rather than to 5–10%. Its drawback is the one already noted — it sees no spillover — so the two calibrations are complementary: the absorber measures the electronics accurately, the Sun measures the whole telescope including everything the absorber bypasses, and the difference between them is an estimate of the spillover term itself.

17.8.4 Method C: cold sky and the survey slope

The third method uses only the sky, and exploits the fact that the 1420 MHz continuum brightness is already known everywhere from the Stockert/Villa-Elisa survey. Point at N directions of differing sky brightness and record the total power:

$$P_i = G(T_0 + T_{\text{sky},i}), \quad (77)$$

where $T_{\text{sky},i}$ is the survey brightness *convolved to this telescope's beam* — which is exactly what the simulator of Section 11 computes — and $T_0 = T_{\text{rx}} + T_{\text{spill}} + T_{\text{atm}}$ collects everything that does not depend on where the dish points. A straight-line fit of P_i against $T_{\text{sky},i}$ then yields both unknowns at once:

$$G = \frac{dP}{dT_{\text{sky}}}, \quad T_0 = \frac{P_{\text{intercept}}}{G}, \quad T_{\text{sys}} = T_0 + T_{\text{sky}}. \quad (78)$$

Its attraction is that it needs no absorber, no ephemeris and no external flux measurement, and that it isolates T_0 — the receiver-plus-spillover term that Method A cannot reach and Method B cannot separate. Its weakness is the lever arm: at 1420 MHz the sky ranges only from about 3.5 K at the coldest high-latitude minimum to perhaps 30 K beam-averaged in the inner plane, so a $\sim 25\text{ K}$ span is being used to calibrate a $\sim 70\text{ K}$ system. The gain must be stable to a few parts in a thousand across the whole set of pointings, which in practice means taking them within a few minutes and at similar elevations so that spillover does not vary underneath the fit.

17.8.5 Recommended procedure

For this telescope, the three methods are complementary rather than alternative, and the sensible order is:

1. Calibrate the pointing first (Section 7.11). Every method below assumes the beam is where it is thought to be.
2. Look up the day's 1415 MHz solar flux.
3. Run a Sun raster and take the fitted peak, immediately followed by a blank-sky reference $5\text{--}10^\circ$ away at the same elevation. Repeat three or four times, alternating, over a few minutes.
4. Compute the SEFD from Equation (69). This is the primary, assumption-free result.
5. Take the same pair as a two-point calibration, Equation (73), to obtain the kelvin-per-unit scale k and hence T_{sys} . Enter that measured value in the receiver's calibration box instead of the assumed 70 K, and use it in Equation (76) for every spectrum of the session.

6. Add a third, intermediate point — the Moon, Cas A or the plane continuum — and confirm the three lie on a straight line through the origin. A non-zero intercept is a detector offset and must be subtracted before Equation (74); curvature means the Sun measurement is compressed and the SDR gain must come down.
7. Run a set of cold-sky pointings spanning as much of the 3.5–30 K range as the sky allows, and fit for G and T_0 . This gives T_{sys} in kelvin independently of the Sun.
8. Combine: the two together over-determine the system, and their ratio yields A_e and hence the aperture efficiency — a genuine measurement of the antenna, rather than the assumed 0.53 used throughout this document.
9. Cross-check against the Moon, not against a fixed radio source. Observe the Moon’s position, wait until it has moved a beamwidth — nine hours, or simply the following night — and repeat on the same celestial coordinates. The difference is the Moon alone, with the Galactic background subtracted identically rather than approximately, and at a predictable 1.20 K it is an independent test of the same scale the Sun gave. Section 17.6 explains why Cassiopeia A cannot serve here.
10. Use Cassiopeia A only as a *stability* test. Repeat the same on/off pair through an observing session: the absolute value is contaminated by the background and means little, but its repeatability at the 0.1 K level is good evidence that the gain is steady enough for the H I work.

Table 32: Uncertainty budget for the solar method

Term	Typical	Reduced by
Solar flux density	5–10%	Same-day 1415 MHz measurement
Pointing	1–3%	Raster peak rather than single pointing
Gain drift between on and off	1–2%	Rapid alternation
Beam coupling f	< 1%	Equation (65)
Atmosphere	1–3%	Observing near the zenith
Detector offset P_0	0–5%	Third calibration point (Section 17.8.3)
Detector non-linearity	0–5%	SDR gain fixed, Sun not clipping
SEFD, combined	6–12%	
Aperture efficiency η_A	10–15%	Method C, or a second calibrator
T_{sys} in kelvin, combined	12–20%	

A system temperature known to 15% is entirely adequate for the science this telescope does: Equation (59) is linear in T_B , so a column density carries the same fractional error, and no 21 cm result from a 3 m dish rests on better than that.

18 The codebase

18.1 Repository and licence

The whole system — both firmwares, the receiver, the scheduler, the simulator, the enclosure and this document — lives in a single repository:

<https://github.com/acrerd/21-cm-radio-telescope-v2>

It is released under the MIT licence, copyright Acre Road Observatory. There is one long-lived branch, `main`; the observatory computer runs directly from a clone of it at `/home/astro/21-cm-radio-telescope-v2`, which is why several launcher scripts contain that path literally.

18.2 Working on GitHub

Outstanding work is recorded in GitHub issues and nowhere else. Two review documents that used to live in `docs/` were folded into issues and deleted; their content remains in the git history. Each issue carries the finding, `file:line` references, a suggested approach and the steps that verify it.

```
1 gh issue list
2 gh issue view 7 --comments
```

Listing 8: Reading the work queue

One label matters operationally. Issues marked **needs-hardware** cannot be closed on a development machine: they need a bench ESP32, the Due emulator of Section 13, or the mount itself, and elsewhere they are compile-check-only. Several issues also record conclusions that took considerable analysis and are easy to re-derive incorrectly — the reasoning is written out in each one, and re-deriving from first principles tends to reach the plausible-but-wrong version.

18.3 Layout

```
1 include/ src/ platformio.ini      Arduino Due drive firmware
2 esp32_controller_arduino/        WT32-ETH01 controller firmware
3 receiver_scheduler/              GNU Radio receiver, Flask scheduler, calibration
4 astro_simulator/                 Desktop sky simulator
5 astro_simulator/web/             Browser port of the same
6 docs/                             Manuals and this article
7 enclosure/                       Printed case for the controller board
8 tools/                           Bench-test utilities
```

Listing 9: Top-level structure

The Due firmware sits at the repository root rather than in a subfolder because PlatformIO expects `src/` and `include/` beside its `platformio.ini`; the controller has its own `platformio.ini` and is therefore a separate project in the same tree.

18.4 Root and drive firmware

Table 33: Repository root and Arduino Due firmware

File	Purpose
README.md	Project overview, architecture sketch and quick start
LICENSE	MIT licence
CLAUDE.md	Conventions and hard-won constraints for anyone — human or AI assistant — editing the code. The prose equivalent of the “do not undo this” notes throughout this document
.gitignore	See Section 18.9
.claude/settings.json	Per-project assistant settings; currently empty
platformio.ini	Due build: environments <code>due</code> and <code>simulation</code> , and the Due-FlashStorage dependency
include/config.h	Every pin, every default, the <code>Config</code> struct persisted to flash, and the simulation parameters
src/main.cpp	The entire drive firmware — state machine, homing, motion profiles, encoders, current sensing, faults, serial interface
test_coordinates.ipynb	Validates the controller’s coordinate routines against <code>astropy</code> ; the origin of the accuracy figures in Section 6

18.5 Controller firmware

Table 34: `esp32_controller_arduino/`

File	Purpose
<code>platformio.ini</code>	Environments <code>wt32-eth01</code> (default), <code>wt32-eth01-ota</code> , legacy <code>esp32s3</code> , and <code>native</code> for host tests
<code>partitions.csv</code>	Single-app flash layout
<code>partitions_wt32_ota.csv</code>	Dual-app layout, required for over-the-air updates
<code>data/index.html</code>	Standalone copy of the web UI. Not compiled in and no filesystem image is built — the served page is <code>src/index_html.h</code>
<code>.vscode/extensions.json</code>	Recommends the PlatformIO extension
<code>src/config.h</code>	Compile-time defaults: pins, ports, observer position, NTP servers, mount limits, horizon, stow
<code>src/main.cpp</code>	Application: the tracking loop, clock discipline, task setup
<code>src/state.h</code>	The single shared <code>SRTState</code> struct
<code>src/sync.{cpp,h}</code>	One recursive mutex covering everything touched by both <code>loopTask</code> and <code>async_tcp</code> , with the rules for using it
<code>src/settings.{cpp,h}</code>	Runtime settings and their NVS persistence
<code>src/coordinates.{cpp,h}</code>	Julian date, sidereal time, precession, alt/az, galactic, Sun, Moon, galactic-plane target
<code>src/pointing.{cpp,h}</code>	Refraction, the pointing model, and the true/drive frame transform — the boundary of Section 7
<code>src/srt_serial.{cpp,h}</code>	UART link to the Due: status parsing, target sending, the serial log
<code>src/web_server.{cpp,h}</code>	The HTTP API of Appendix C
<code>src/index_html.h</code>	The web interface, embedded as one string
<code>src/stellarium.{cpp,h}</code>	Stellarium telescope protocol on TCP 10001
<code>src/wifi_manager.{cpp,h}</code>	Access point and station management, credential storage
<code>test/shim/Arduino.h</code>	Host stand-in for the few Arduino facilities <code>pointing.cpp</code> uses
<code>test/shim/Preferences.h</code>	Host stand-in for NVS
<code>test/shim/shim.cpp</code>	Implementations behind those two headers
<code>test/test_coordinates/test_main.cpp</code>	Host tests of the coordinate transforms
<code>test/test_pointing/test_main.cpp</code>	Host tests of the pointing model and the frame round trip

18.6 Receiver, scheduler and calibration

Table 35: receiver_scheduler/

File	Purpose
README.md	Receiver and scheduler documentation
requirements.txt	Flask, PyEphem, pytest — what radioconda does not already supply
b210_h1_receiver.py	GNU Radio spectrometer: the two-product headless recorder of Section 10.1.1, the console window, HDF5 logging, the live sidecar
tuning.py	The fixed instrument — LO, rate, gain, the two bands — shared by scheduler and receiver so they cannot disagree; the older LO-offset planner for the console window
h1_web_scheduler.py	The Flask scheduler: schedule, observation execution, telescope control, firmware updates, the calibration, horizon and observe endpoints
observation_files.py	Where recordings go and what they are called
observation_plot.py	Reading a recording (live or finished, either product, always as counts), the spectrum, drift and check plots, the LSR axis
bandpass.py	The measured instrument response: one template per product, fitted on the Lockman Hole
rf_calibration.py	The gain and T_{sys} fit against the simulator’s sky, candidate fields, the clock-shift rule
drift_fit.py	The total-power fit of a drift scan against the predicted drift curve, continuum band only
drift_park.py	Parking a drift scan on the drive grid: the controller’s transform in Python, validated against its own reports
sun_scan.py	Sun raster, Gaussian fit, pointing-model fit, quality gates and the controller wire format
horizon_scan.py	The strip scan of Section 10.4.5
web/	The operator page: <code>index.html</code> , <code>app.css</code> , twelve classic scripts under <code>js/</code>
test_*.py, conftest.py	The suite of Section 13; the isolation fixtures
pointing_data.json	Accumulated Sun-scan records. The tracked set holds 23 scans from July and August 2026, all at <code>record_version 1</code> — residuals against a model that is no longer knowable, so the fit skips them all (Section 7). They are kept for their measured beam widths, a median of 5.1° , which corroborate the 4.9° of Section 9.1
read_h1_data.ipynb	Reads HDF5 files, plots waterfalls and mean spectra
start_scheduler.sh	Launches the scheduler, reusing an already-serving instance
start_srt_software.sh	Desktop launcher for the whole software set
start_platformio_monitor.sh	Opens a serial monitor on the Due
SRT Software.code-workspace	VS Code workspace whose <code>folderOpen</code> task starts the scheduler

18.7 Sky simulator

Table 36: `astro_simulator/` — desktop version

File	Purpose
<code>README.md</code>	Usage, data provenance and the compression scheme
<code>requirements.txt</code>	NumPy, SciPy, matplotlib, astropy
<code>astro_simulator.py</code>	The interactive simulator of Section 11
<code>instrument.py</code>	The one copy of the measured numbers: the surveyed site, the measured beam, the total-power instability, the main-beam efficiency
<code>horizon_store.py</code>	The horizon archive of Section 10.4.5 — on the simulator side because that is the layer everything may depend on
<code>hi4pi_compact.npz.xz</code>	The HI4PI cube reduced from 33 GiB to ~23 MB
<code>hi4pi_compress.py</code>	Regenerates that file from the full survey
<code>hi4pi_data.py</code>	Download helper for the full cube from CDS
<code>continuum_1420_compact.npz.xz</code>	Compact 1420 MHz continuum sky, ~0.7 MB
<code>continuum_compress.py</code>	Regenerates it from the Stockert/Villa-Elisa survey
<code>nhi_grid_cache.npy</code>	Precomputed N_{HI} display map, so the window opens instantly
<code>run.sh</code> , <code>run.bat</code>	Launchers passing arguments through
<code>.gitignore</code>	Excludes the downloaded full-survey files

The browser port exists so the simulator can be used with no installation at all — a teaching consideration — and is deliberately restricted to the compact datasets, with no full-cube code paths.

Table 37: `astro_simulator/web/` — browser version

File	Purpose
<code>README.md</code>	How to serve and use it
<code>PLAN.md</code>	Design record and validation strategy
<code>index.html</code>	The page itself
<code>package.json</code>	Exists only so <code>node</code> runs the test harness; there is no build step
<code>js/main.js</code>	Bootstrap: fetch and decompress the bundles, wire the UI, read URL parameters in place of CLI flags
<code>js/skydata.js</code>	Sky and spectrum engine; a line-faithful port of the desktop <code>DishSimulator</code>
<code>js/coordinates.js</code>	Coordinate and time routines, ported from <code>coordinates.cpp</code>
<code>js/ephemeris.js</code>	Sun and Moon, plus the velocity-frame offsets
<code>js/map.js</code>	Mollweide all-sky canvas with overlays and click-to-point
<code>js/plot.js</code>	Spectrum and drift-scan panel
<code>js/ui.js</code>	Parameter row, targets menu, frame cycling, the band per map type, <i>Reset</i> , <i>Schedule</i>
<code>js/clock.js</code>	The pinnable clock
<code>make_web_data.py</code>	Converts the compact datasets into the web bundles
<code>data/hi4pi_web.bin.gz</code>	H I bundle
<code>data/continuum_web.bin.gz</code>	Continuum bundle
<code>data/meta.json</code>	Bundle geometry and scaling
<code>gen_golden.py</code>	Runs the desktop simulator at a fixed epoch to generate reference values
<code>test/golden.json</code>	Those reference values
<code>test/run_golden.mjs</code>	Checks the JavaScript engine reproduces every one of them

The golden-vector arrangement is the part worth noticing: the two simulators are independent implementations, and a fixed-epoch comparison is what stops them drifting apart.

18.8 Documentation, enclosure and tools

Table 38: docs/, enclosure/ and tools/

File	Purpose
docs/SRT_Technical_Article.tex	This document
docs/SRT_Technical_Article.pdf	Its built form, tracked so it can be read without a L ^A T _E X installation
docs/.latexmkrc	Writes <code>version.tex</code> before every <code>latexmk</code> run, so the build stamp cannot be forgotten
docs/update_version.sh	The same stamp by hand, for builds that do not go through <code>latexmk</code>
docs/SRT_DRIVE_MANUAL.md	Operations manual for the drive firmware
docs/SRT_DRIVE_MANUAL.html	Its rendered form
docs/ESP32_CONTROLLER.md	Controller technical manual
docs/WT32_ETH01_MIGRATION.md	The ESP32-S3 to WT32-ETH01 migration, including the serial-flash recovery procedure
docs/OBSERVATORY_HOST_SETUP.md	Building the observatory host and its private link, Section 8
enclosure/wt32_eth01_enclosure.scad	Parametric OpenSCAD source for the controller case
enclosure/*.stl	Exported solid, base and lid
tools/due_emulator.py	The Due protocol emulator of Section 13

18.9 What is deliberately not tracked

Some absences are decisions rather than oversights, and each has cost someone time at least once.

Table 39: Untracked files that matter

Path	Why not tracked
receiver_scheduler/scheduler_config.json	Runtime configuration, and site-specific. It is the source of truth for the controller URL
receiver_scheduler/h1_schedule.json	The scheduler rewrites it on every edit and recreates it empty if absent. Ignored in both its real location and the repository root, where a stray copy appears if the scheduler is ever launched from there
receiver_scheduler/pointing_model.json	Refitted from <code>pointing_data.json</code> whenever it is needed
receiver_scheduler/scheduler.log*	Rotating log
receiver_scheduler/data/, *.h5	Observation data; large, and not source
wifi_creds.json	Credentials
docs/version.tex	Generated build stamp. Committing it would mean the document claims a commit hash that goes stale on the next commit
.pio/, *.aux, *.idx, ...	Build artefacts and L ^A T _E X intermediates

19 Conclusion

The SRT drive controller is a layered system in which each layer holds exactly one kind of knowledge: the Due knows only pulses and motors, the controller knows the sky and the transform between the sky and the mount, and the scheduler knows what is to be observed and when. The two coordinate frames, and the single file where they meet, are the organising idea; most of the failures worth documenting here have come from mixing them.

Current capabilities:

- 0.5° encoder resolution with a fitted pointing model that removes the residual mount errors and a refraction correction applied as physics
- Absolute, model-free calibration data, so every fit stands on its own rather than being composed onto its predecessor
- J2000 coordinates throughout, compatible with modern catalogues and planetarium software
- Multiple control paths: web UI, REST API, Stellarium and the scheduler
- Automated scheduling, drift scans, satellite tracking and unattended data acquisition
- A private, reversible network link that keeps the controller's address and its clock under the observatory's own control
- Four hardware-free test paths — drive simulation, host unit tests, pytest and a Due emulator — covering everything but the motors
- A survey-backed sky simulator that predicts what the instrument should see, and can hand its pointing to the real dish

The instrument's principal remaining gap is calibration rather than control: the velocity axis is topocentric and the temperature scale rests on an assumed system temperature. Both are described in Sections 9 and 16, and both are addressed in analysis rather than at the telescope.

A Pin quick reference

```

1 Motor control: 8(PWM-Az), 9(DIR-Az), 10(PWM-Alt), 11(DIR-Alt)
2 Driver reset: 22(Az), 24(Alt)
3 Encoders: 12(Az), 13(Alt)
4 Fault flags: 5,4(Az FF1,FF2), 7,6(Alt FF1,FF2)
5 Current sense: A0(Alt), A1(Az)
6 Serial1: 18(TX -> WT32 I014), 19(RX <- WT32 I04)
7 Calibrator: 26

```

Listing 10: Arduino Due pin summary

B Command quick reference

```

1 45 180           # Slew to Alt=45, Az=180
2 HOME            # Run the homing sequence
3 STOP            # Emergency stop
4 RESET           # Clear fault
5 CAL ON          # Calibrator noise source on
6 STATUS          # One status line now
7 CONFIG          # Show configuration
8 SET AZMAX 350    # Set a parameter
9 SET DEBOUNCE 50  # Azimuth encoder debounce (ms)
10 SAVE           # Save to flash
11 HELP           # Show all commands

```

Listing 11: Common Due serial commands

C HTTP API quick reference

C.1 Controller API

All endpoints are HTTP GET unless noted. Base URL `http://192.168.50.120`.

C.1.1 Status and information

Endpoint	Description
<code>/ping</code>	Minimal liveness check
<code>/network</code>	Ethernet and DNS diagnostics
<code>/status</code>	Drive <code>alt/az</code> , <code>true_alt/true_az</code> , RA/Dec, galactic <code>l/b</code> , target, currents, status, fault, <code>is_slewing</code> , calibrator, <code>pointing_loaded</code> , <code>clock_state</code> , <code>clock_age_s</code> , <code>lock_timeouts</code> , <code>malformed_status</code> , <code>last_homing</code> (the encoder error at the last homing, issue #24)
<code>/tracking</code>	Tracking state: enabled, RA/Dec, target name, axis mode, offsets, waiting flags
<code>/ephemeris</code>	Sun, Moon and galactic-plane (<code>plane</code>) positions
<code>/pointing</code>	The resident pointing model
<code>/serial/log</code>	Serial communication log
<code>/time/status</code>	<code>synced</code> , <code>source</code> , <code>sync_state</code> , <code>stale</code> , <code>last_sync_age_s</code> , <code>last_offset_ms</code> , <code>sync_count</code> , <code>utc</code>
<code>/settings</code>	All settings (Table 12)

C.1.2 Telescope control

Endpoint	Parameters	Description
<code>/direct</code>	<code>alt</code> , <code>az</code>	True alt/az slew; disables tracking
<code>/goto</code>	<code>ra</code> (h), <code>dec</code>	One-time slew to J2000 RA/Dec
<code>/goto/galactic</code>	<code>l</code> , <code>b</code>	One-time slew to galactic
<code>/track/radec</code>	<code>ra</code> (h), <code>dec</code>	Track J2000 RA/Dec
<code>/track/galactic</code>	<code>l</code> , <code>b</code>	Track galactic coordinates
<code>/track/galactic-plane</code>	—	Track the plane at the acquisition floor
<code>/track/sun</code>	—	Track the Sun
<code>/track/moon</code>	—	Track the Moon
<code>/tracking/enable</code>	<code>enable</code> (0/1)	Enable or disable tracking
<code>/tracking/axis</code>	<code>mode</code> = <code>both</code> , <code>az</code> or <code>alt</code> ; <code>alt</code> , <code>az</code>	Track both axes or hold one fixed
<code>/stop/movement</code>	—	Stop motion, pause tracking commands 10s
<code>/stop/slewing</code>	—	Stop the current slew
<code>/stop/tracking</code>	—	Stop tracking, leave the mount
<code>/stop/all</code>	—	Stop motion and clear the tracking target
<code>/home</code>	—	Run the Due physical homing sequence
<code>/go-home</code>	—	Slew to the stow position (drive frame)
<code>/reset</code>	—	Clear an active Due fault
<code>/calibrator</code>	<code>on</code> (0/1)	Noise source
<code>/offset</code>	<code>alt</code> , <code>az</code>	Sky-frame pointing offset
<code>/offset/clear</code>	—	Clear the offset

C.1.3 Pointing, time, network and settings

Endpoint	Parameters	Description
/pointing/apply	POST, JSON document	Replace and persist the model
/pointing/clear	—	Zero the model and erase NVS
/time/status	—	Clock health
/time/sync	—	Request an NTP poll now
/time/set	timestamp	Set the clock from the browser
/wifi/status	—	Ethernet and WiFi status, IPs, MACs
/wifi/scan	—	Scan for networks
/wifi/connect	ssid, password	Join a network
/wifi/forget	—	Clear saved credentials
/wifi/power	enable (0/1)	WiFi radio on/off
/eth/save	dhcp, ip, gateway, subnet, dns	Ethernet config (reboot to apply)
/settings/save	see Table 12	Save settings
/settings/reset	—	Reset settings to compiled defaults (the pointing model is <i>not</i> affected)

C.2 Scheduler API

Served on `http://127.0.0.1:5000`.

Endpoint	Method	Description
/api/schedule	GET/POST	Read or save the schedule; clear
/api/status	GET	Running state, current observation
/api/start	POST	Start an observation immediately
/api/stop	POST	Stop the current observation
/api/stop_all	POST	Stop everything, including calibration
/api/receiver/status	GET	Whether a manual or observation receiver
/api/receiver/start	POST	Manually boot the receiver for observation
/api/telescope	GET	Controller status, proxied
/api/config	GET/POST	Scheduler configuration (known as <code>config</code>)
/api/fetch_tle	POST	Fetch a TLE from CelesTrak
/api/predict_pass	POST	Predict the next satellite pass
/api/drift_preview	GET	Drift-scan pointing and next drift-scan
/api/instrument	GET	The fixed instrument in force, or the current instrument if not hidden
/api/observations	GET	The recordings catalogue: recording ID, instrument, mode, units, whether still recording
/api/observe/info	GET	A recording's facts for the detector
/api/observe/plot	GET	Plot a recording, live or finished
/api/observe/live	GET	The live band-power trace of the recording, or the scan or solar track
/api/observe/fit	POST	Fit gain and T_{sys} to a recording's spectrum, total-power for a detector
/api/observe/fit/plot, /api/observe/fit/apply	GET, POST	Draw the proposal; adopt it as the recording's fit (for a total-power fit)
/api/observe/download	GET	The recording as an attachment
/api/simulator/schedule	POST	Book, or start now, an observation with the simulator's state
/api/rf/status, /api/rf/target	GET	RF calibration state; candidate frequencies above the horizon
/api/rf/run, /api/rf/cancel	POST	Run the bandpass or gain job
/api/rf/goto	POST	Track the Lockman Hole (or other target) for pass measurement
/api/rf/bandpass/plot, /api/rf/gain/plot	GET	The check plots
/api/horizon/start, /api/horizon/stop, /api/horizon/status	POST, POST, GET	The horizon scan
/api/horizon/profile, /api/horizon/profiles, /api/horizon/active	GET, GET, POST	The profile in force; the archive of profiles
/api/log	GET	Last N lines of the scheduler log
/api/firmware/update	POST	Run the OTA firmware upload
/api/firmware/status	GET	Firmware upload progress
/api/sunscan/start	POST	Run one Sun scan

Endpoint	Method	Description
/api/sunscan/stop	POST	Cancel the running scan
/api/sunscan/status	GET	Scan progress and result
/api/sunscan/image	GET	Last scan image
/api/calday/start	POST	Start a calibration day
/api/calday/stop	POST	Stop the calibration day
/api/calday/status	GET	Phase, scans completed, next
/api/calday/data	GET	Accumulated scan records
/api/calday/clear	POST	Discard accumulated scan records
/api/calday/fit	POST	Fit the pointing model and plot
/api/calday/plot	GET	The calibration-day plot
/api/calday/apply	POST	Push the model to the controller
/api/calday/model	GET	The last fitted model

C.3 Examples

```

1 # Track RA/Dec (J2000), galactic coordinates, the plane, the Sun
2 curl "http://192.168.50.120/track/radec?ra=12.5&dec=-30"
3 curl "http://192.168.50.120/track/galactic?l=120&b=0"
4 curl "http://192.168.50.120/track/galactic-plane"
5 curl "http://192.168.50.120/track/sun"
6
7 # Direct alt/az slew (true frame), and park at stow (drive frame)
8 curl "http://192.168.50.120/direct?alt=45&az=180"
9 curl "http://192.168.50.120/go-home"
10
11 # Stop tracking; read status, ephemeris and clock
12 curl "http://192.168.50.120/tracking/enable?enable=0"
13 curl "http://192.168.50.120/status"
14 curl "http://192.168.50.120/ephemeris"
15 curl "http://192.168.50.120/time/status"
16
17 # Pointing model: read, apply, clear
18 curl "http://192.168.50.120/pointing"
19 curl -X POST "http://192.168.50.120/pointing/apply" \
20   -H "Content-Type: application/json" \
21   -d '{"version":1,"terms":{"IE":-0.72,"IA":1.98,"AN":0.31,"AE":-0.10}}'
22 curl "http://192.168.50.120/pointing/clear"

```

Listing 12: Controller API examples

```

1 # Schedule and status
2 curl "http://127.0.0.1:5000/api/schedule"
3 curl "http://127.0.0.1:5000/api/status"
4
5 # A calibration day, then fit and apply the model
6 curl -X POST "http://127.0.0.1:5000/api/calday/start" \
7   -H "Content-Type: application/json" \
8   -d '{"interval_minutes": 30, "sdr_type": "b210"}'
9 curl "http://127.0.0.1:5000/api/calday/status"
10 curl -X POST "http://127.0.0.1:5000/api/calday/fit"
11 curl -X POST "http://127.0.0.1:5000/api/calday/apply"
12
13 # Drift-scan preview
14 curl "http://127.0.0.1:5000/api/drift_preview?frame=radec&coord1=5.58&coord2=22.0&time=21:30"

```

Listing 13: Scheduler API examples

References

- [1] J. Meeus, *Astronomical Algorithms*, 2nd ed., Willmann-Bell, 1998. Source of the solar and lunar ephemerides of Section 6.
- [2] J. H. Lieske, T. Lederle, W. Fricke and B. Morando, “Expressions for the precession quantities based upon the IAU (1976) system of astronomical constants”, *Astronomy & Astrophysics*, **58**, 1–16, 1977. The precession angles ζ_A , z_A , θ_A .
- [3] S. Aoki, B. Guinot, G. H. Kaplan, H. Kinoshita, D. D. McCarthy and P. K. Seidelmann, “The new definition of Universal Time”, *Astronomy & Astrophysics*, **105**, 359–361, 1982. The GMST expression used for sidereal time.
- [4] G. G. Bennett, “The calculation of astronomical refraction in marine navigation”, *Journal of Navigation*, **35**(2), 255–259, 1982. The refraction formula of Equation (28), here scaled by 1.15 for radio wavelengths.
- [5] P. T. Wallace, “A rigorous algorithm for telescope pointing”, *Proc. SPIE*, **4848**, 125–136, 2002. The alt-az pointing model terms IE, IA, AN, AE, CA, NPAE and TF.
- [6] HI4PI Collaboration, “HI4PI: a full-sky H I survey based on EBHIS and GASS”, *Astronomy & Astrophysics*, **594**, A116, 2016. The H I sky used by the simulator of Section 11.
- [7] P. Reich and W. Reich, “A radio continuum survey of the northern sky at 1420 MHz”, *Astronomy & Astrophysics Supplement*, **63**, 205–292, 1986; extended to the southern sky at Villa-Elisa by Reich, Testori and Reich, 2001. The 1420 MHz continuum sky used by the simulator.
- [8] H. I. Ewen and E. M. Purcell, “Observation of a line in the galactic radio spectrum”, *Nature*, **168**, 356, 1951. The detection of the 21 cm line.
- [9] T. L. Wilson, K. Rohlfs and S. Hüttemeister, *Tools of Radio Astronomy*, 6th ed., Springer, 2013. Source of the antenna and radiometry relations of Sections 9 and 17, and of the column-density constant in Equation (59).
- [10] J. W. M. Baars, R. Genzel, I. I. K. Pauliny-Toth and A. Witzel, “The absolute spectrum of Cas A”, *Astronomy & Astrophysics*, **61**, 99–106, 1977. The flux scale for the calibrator sources of Table 29.
- [11] A. S. Trotter et al., “The secular decrease of Cassiopeia A”, *Monthly Notices of the Royal Astronomical Society*, **469**, 1299, 2017. The $0.670 \pm 0.019\%$ per year L-band fading rate applied to Cas A.
- [12] US Air Force Radio Solar Telescope Network (RSTN) and the Dominion Radio Astrophysical Observatory solar flux monitoring programme. Daily solar flux densities at 1415 MHz — within 0.4% of the observing frequency, and the value required by Equation (69).
- [13] F. R. Hoots and R. L. Roehrich, “Models for propagation of NORAD element sets”, *Spacetrack Report No. 3*, 1980. The SGP4 propagator behind the scheduler’s satellite tracking.
- [14] Standards of Fundamental Astronomy (SOFA) Board, *SOFA Astrometry Tools*; and its ERFA fork, as used through `astropy` for the barycentric velocity correction of Section 9.5 and as the reference against which the on-board transforms are checked.

Index

- 21 cm line, *see* H I line
- accuracy, pointing, 22
- ADC clipping, 60
- address plan, 29
- AE (azimuth axis tilt, east), 23
- ambient absorber, 58
- AN (azimuth axis tilt, north), 23
- antenna temperature, 56
- antenna theorem, 55
- aperture, 32
- aperture efficiency, 55
- architecture, 7
- Arduino Due, 8
 - pin assignments, 9, 69
- atmosphere, 58
- atmospheric attenuation, 59
- atmospheric opacity, 58
- automatic gain control, 60
- azimuth coverage, 26
- azimuth wrap-around, 20
- B210
 - clock, 37
- backlash compensation, 15
- bandpass correction, 60
- bandpass template, 35
- bandwidth, 55
- barycentric correction, 36
- baud rate, 8
- beam, 32
 - averaging over the Galaxy, 55
 - degeneracy with gain, 33
 - FWHM, 32
 - measured, 33
 - solid angle, 32
- beam coupling factor, 56
- beam dilution, 32
- blank-sky calibration, 34
- branch, `main`, 62
- brightness temperature, 54
- building
 - controller firmware, 48
 - drive firmware, 48
- CA (collimation error), 23
- calibration, 23
 - recommended procedure, 61
 - two-point, 59
- calibration day, 28
- calibrator, 34
 - recorded in metadata, 34
- calibrators
 - unusable at low resolution, 56
- Cassiopeia A, 56
 - secular fading, 56
- CelesTrak, 43
- channel averaging, 55
- `CLAUDE.md`, 63
- clock, 18
 - error as pointing error, 18
- clock offset, 37
- codebase, 62
- cold sky calibration, 61
- column density, 54
- communication interfaces, 8
- condition number, 26
- configuration
 - drive firmware, 15
 - persistence, 15
- confusion, background, 56
- continuum sky, 46
- controller firmware, 16
- coordinate frames, 6
 - drive, 6
 - true, 6
- coordinate transformations, 21
- CORS, 41
- counterfeit hardware, 29
- covariance, 26
- cross-elevation, 25
- cross-task locking, 16
 - `sync.cpp`, 64
- current sensing, 10
 - filtering, 10
 - hall-effect sensors, 10
- data flow, 8
-
- datum, absolute (T, D), 25
- deadband, 17
- design matrix, 25
- detector offset, 59
- DHCP, 30
- Dicke switching, 34
- direction inversion, 9
- DNS, 19
- `dnsmasq`, 30
- drift scan, 43
 - drive grid, 27
 - total-power fit, 35
- drive firmware, 11
- drive frame, *see* coordinate frames
- `driveToTrue()`, 24
- Due emulator, 50
- `DueFlashStorage`, 15
- dynamic range, 56
- effective area, 55
- Einstein A coefficient, 54
- enclosure, printed, 11
- encoder counts, lost, 27
- encoder origin, 13
- encoder quantum, 27
- encoders, 10
 - debounce, 10
 - reed switch, 10
- ESP32 controller, 10
- `ESPAsyncWebServer`, 16
- `esptools`, 31
- Ethernet, 18

- Ettus B210, [37](#)
- fault flags, [9](#)
- fault injection, [50](#)
- faults, [16](#)
 - detection, [16](#)
- firewall, [30](#)
- fixed instrument, [37](#)
- fixed-point iteration, [24](#)
- flash partitions, [64](#)
- forbidden transition, [54](#)
- FT232 programmer, [10](#)
- FWHM, [25](#)
- gain calibration, [35](#)
- gain instability, [34](#)
- gain, receiver, [59](#)
- Galactic constants R_0 , V_0 , [55](#)
- Galactic continuum
 - as a contaminant, [56](#)
- galactic coordinates, [22](#)
- Galactic longitude, [55](#)
- galactic plane, [20](#)
 - acquisition floor, [20](#)
- galactic rotation, [55](#)
- Gaussian fit, [25](#)
- geometric area, [55](#)
- gh CLI, [63](#)
- GitHub
 - issue workflow, [63](#)
 - issues, [63](#)
 - labels, [63](#)
 - repository, [62](#)
- GitHub issues, [53](#)
 - needs-hardware, [50](#)
- .gitignore, [68](#)
- GMST, [21](#)
- GNU Radio, [37](#)
- golden vectors, [67](#)
- ground pickup, *see* spillover
- H I line, [54](#)
 - hyperfine transition, [54](#)
- H-bridge drivers, [9](#)
- hardware, [8](#)
- HDF5, [39](#)
 - attributes, [39](#)
 - datasets, [39](#)
 - SWMR, [39](#)
- HI4PI survey, [46](#)
 - compact cube, [46](#)
- homing, [13](#)
 - before an observation, [27](#)
 - three-phase sequence, [13](#)
- horizon
 - archive, [43](#)
 - measured profile, [43](#)
- horizon scan, [43](#)
- horizon, observing, [20](#)
- horizontal coordinates, [22](#)
- hot/cold load, [58](#)
- hour angle, [22](#)
- HTTP API, [19](#), [70](#)
- hydrogen, neutral, *see* H I line
- IA (azimuth index error), [23](#)
- ICRS, [21](#)
- IE (elevation index error), [23](#)
- installation, [48](#)
- integration time, [33](#)
- IP addresses, [29](#)
- J2000, [21](#)
- Julian date, [21](#)
- kelvin scale, [59](#)
- kinematic distance, [55](#)
- known limitations, [53](#)
- least squares, [25](#)
 - weighted, [25](#)
- level populations, [54](#)
- licence, [62](#)
- limit switches, [11](#)
- linearity, [59](#)
- live plot, [35](#)
- local sidereal time, [21](#)
- Lockman Hole, [35](#)
- low-noise amplifier, [33](#)
- LSR (Local Standard of Rest), [36](#)
- lwIP, [19](#)
- main-beam efficiency, [32](#)
- mDNS, [18](#)
- MIT licence, [62](#)
- Moon
 - as calibrator, [60](#)
 - ephemeris, [22](#)
- motion profiles, [13](#)
- motors, [9](#)
- mount limits, [11](#)
- NAT, [30](#)
- native environment, [49](#)
- network card, [29](#)
- networking, [18](#)
- NetworkManager, [30](#)
- neutral hydrogen, [54](#)
- nmcli, [30](#)
- noise diode, [34](#)
- noise figure, [33](#)
- noise, radiometer, [33](#)
- NPAE (axis non-perpendicularity), [23](#)
- NTP, [18](#)
- NVS, [17](#)
 - pointing namespace, [24](#)
- observation parameters, [44](#)
- observations folder, [38](#)
- observatory host, [29](#)
- observer location, [48](#)
- observing, [51](#)
- on/off ratio, *see* position switching
- operation, [50](#)
- optical depth, [54](#)
- OTA updates, [18](#), [31](#), [48](#)
- outstanding work, [53](#)
- overcurrent, [10](#)
- overflow, [38](#)

- parameter significance, 26
- pin assignments, 9, 69
- PlatformIO, 48
 - project layout, 63
- pointing error
 - in calibration, 59
- pointing model, 23
 - clamps, 24
 - design, 23
 - in force, 27
 - persistence, 24
 - terms, 23
 - wire format, 28
- pointing offset, operator, 18
- position switching, 60
- precession, 21
- private link, 29
- process lock, 41
- PULSES_PER_DEGREE, 10
- PWM, 13
 - inverted convention, 13
- PyEphem, 43
- pytest, 45
- quality gates, 26
- radial velocity, 55
- radio velocity convention, 36
- radioconda, 41, 49
- radiometer equation, 33
 - practical form, 34
- radiometry, 32
- ramps
 - acceleration, 13
 - deceleration, 13
- raster scan, 25
- Realise button, *see* Schedule button
- receiver, 37
 - headless, 37
- record version, 25
- recordings
 - naming, 38
- recovery, 31
- reduced chi-squared, 26
- refraction, 23
 - Bennett's formula, 23
- repository, 62
 - layout, 63
- respect local horizon, 43
- rotation curve, 55
- RSTN, 58
- RTL-SDR, 37
- runtime state, 68
- safety features, 51
- satellites, 43
- Schedule button, 47
- scheduler, 40
 - API, 71
 - loopback binding, 8
 - web interface, 40
- SDR, 37
- security, 8
- SEFD, 55
- sensitivity
 - K per jansky, 55
- serial commands, 14, 69
- SET command, 15
- settings, controller, 17
- sfu, *see* solar flux unit
- SGP4, 43
- sidereal time, 21
- simulation mode, 16
- sky brightness, 61
- sky simulator, 6, 46
 - browser version, 66
 - clock, 47
 - measured horizon, 47
 - validation, 67
- software limits, 11
- solar flux density, 58
- solar flux unit, 59
- spillover, 33
- spin temperature, 54
- SRTState, 16
- stall detection, 14
- startup sequence, 50
- state machine, 12
- states
 - DRIVING, 12
 - FAULT, 12
 - HOMING, 12
 - IDLE, 12
 - INIT, 12
- status line, 15
 - de-duplication, 15
 - format, 15
 - rate limiting, 15
- Stellarium, 19
- Stockert survey, 46
 - as a calibrator, 61
- stow position, 20
- Sun
 - active, 59
 - as calibrator, 58
 - ephemeris, 22
 - quiet, 59
- Sun scan, 25
- SWMR, 39
- system overview, 6
- system temperature, 33
 - expected values, 58
 - measurement, 58
- systemd, deliberately not used, 42
- tangent-point method, 55
- temperature calibration, 34
- terminal velocity, 55
- test shim, 49
- testing, 49
- TF (tube flexure), 23
- time synchronisation, 18
- TLE, 43
- topocentric velocity, 36
- total power measurement, 58
- TP-Link TG-3468, 29
- tracking, 20
- troubleshooting, 51

- coordinates, [52](#)
- motors, [51](#)
- network, [52](#)
- pointing, [52](#)
- position, [52](#)
- startup, [51](#)
- true frame, *see* coordinate frames
- `trueToDrive()`, [24](#)
- tuning, [37](#)
- two products, [37](#)
- two-point calibration, [59](#)

- UART link, [8](#)
 - line splicing, [50](#)
- ufw, [30](#)
- uncertainty budget, [61](#)
- unit tests, [49](#)
- units in the dataset name, [39](#)
- untracked files, [68](#)

- velocity frames, [36](#)
- velocity resolution, [55](#)
- verification, network, [31](#)

- waypipe, [32](#)
- web interface, [51](#)
- WiFi access point, [18](#)
- WT32-ETH01, [10](#)
 - wiring to the Due, [10](#)

- Y-factor, [34](#), [58](#)